PIE CORP LIMITED

PORTFOLIO IMMORTALIZED

TECHNICAL DESIGN DOCUMENT

Mobile-first digital legacy custody, verification and controlled-release platform

DOCUMENT CONTROL	VALUE
Document owner	Pie Corp Limited
Version	1.0 - Comprehensive draft for engineering and security review
Date	15 October 2026
Status	Design baseline - not yet approved for production
Primary clients	React Native mobile app (Android/iOS) and Next.js web application
Proposed stack	TypeScript, Next.js, Expo/React Native, Prisma, PostgreSQL/Supabase, Better Auth, Brevo, private object storage, optional Cloudinary and AI gateway
Source baseline	Portfolio Immortalized - Design Blueprint.pdf (20-page user-provided blueprint)
Audience	Product owner, engineering, security, QA, operations, privacy/legal advisers

Document purpose and how to use it

This Technical Design Document (TDD) translates the existing Portfolio Immortalized design blueprint and the product-owner amendments into a build-oriented technical specification. It is intended to guide architecture, implementation, validation, release and ongoing operation. It is detailed by design, but it is not a substitute for legal advice, an independent cryptographic design review, or formal security testing.

IMPORTANT DESIGN POSITION This TDD does not authorize automatic AI death determination or automatic release based on inactivity. Inactivity starts a review process. Release requires verified evidence, policy checks and independent human authorization.

Confidentiality: This document describes a proposed system for handling highly sensitive legacy and identity data. Share only with people who need it for design, implementation, legal review, security review or operations.

Requirement-source labels

Label	Meaning	How to interpret it
BASELINE	Present in the uploaded blueprint	Preserved unless explicitly superseded by the revised requirements.
OWNER REQUIREMENT	Explicitly requested in the conversation	Treat as product requirement pending final acceptance criteria.
DESIGN RECOMMENDATION	Proposed by this TDD	Use as the recommended baseline; record deviations as Architecture Decision Records (ADRs).
OPEN DECISION	Cannot safely be finalized from current inputs	Must be resolved by the owner, security lead or legal adviser before the dependent production milestone.
LEGAL REVIEW	Depends on jurisdiction or legal interpretation	Requires qualified legal review and documented approval before production.

Change summary from the original blueprint

Original blueprint item	Revised direction
Python / FastAPI backend; React web UI; Flutter or React Native mobile; PostgreSQL; Fernet	TypeScript-based stack; Next.js web; React Native + Expo mobile; Prisma; PostgreSQL on Supabase; vetted AEAD + envelope key-management design.
Long onboarding with ID, address, recovery contacts and trustees	Minimal registration; collect further profile and legacy-plan data progressively.
Manual “I Am Alive” button and pulse-check timer	Removed. Meaningful authenticated activity may reset an inactivity clock, but inactivity only creates alerts and investigation.
Two mechanisms to trigger death release	Preserve both event-driven/manual reporting and inactivity escalation, while requiring evidence review and authorization for every final release.
Single admin review implied in flow	Dedicated Admin Console, named roles, emergency holds and two-person final release authorization.
Digital legacy document storage and release	Expanded PDF/image capture, multi-page scans, existing or recorded video, audio notes, per-beneficiary assignment, integrity and versions.
Blockchain mention in introduction	Not included by default. Append-only audit records, file hashes and independently protected logs provide a simpler baseline; blockchain requires a separately justified ADR.
AI not specified	Governed AI Gateway for low-risk assistance and reviewed document extraction; never allowed to decide death, approve release or modify authorization directly.

Contents

PART I - PRODUCT AND REQUIREMENTS

- 01 Executive Summary
- 02 Product Scope and Assumptions
- 03 Personas, Roles and Trust Boundaries
- 04 Functional Requirements and Acceptance Criteria
- 05 Non-Functional Requirements and Quality Attributes

PART II - EXPERIENCE AND WORKFLOWS

- 06 Information Architecture and Navigation
- 07 Mobile App Screen Specification
- 08 Web Application and Portal Specification
- 09 Admin Console Specification
- 10 Core User Workflows
- 11 Inactivity, Death Verification and Release State Machine
- 12 File Capture, Upload and Asset Lifecycle

PART III - ARCHITECTURE AND DATA

- 13 System Architecture
- 14 Technology Stack and Repository Layout
- 15 Runtime Components, Jobs and Integrations
- 16 Database Architecture and Data Model
- 17 Data Dictionary and Constraints
- 18 API Design and Endpoint Catalogue
- 19 Eventing, Notifications and Idempotency

PART IV - SECURITY, PRIVACY AND AI

- 20 Security Architecture and Threat Model
- 21 Authentication, Sessions and Device Security
- 22 Authorization, RLS and Administrator Controls
- 23 Cryptography, Key Management and Recovery
- 24 File Storage, Delivery and Cloudinary Decision
- 25 Privacy, Retention and Legal Readiness
- 26 AI Integration Architecture and Use Cases
- 27 AI Safety, Governance and Evaluation

PART V - DELIVERY AND OPERATIONS

- 28 Error Handling and User Feedback
- 29 Audit Logging, Observability and Alerting
- 30 Backup, Disaster Recovery and Continuity
- 31 Testing Strategy and Acceptance Matrix
- 32 CI/CD, Environments and Release Management
- 33 Operational Runbooks and Incident Response
- 34 Performance, Accessibility and Mobile Quality
- 35 App Store and Play Store Readiness
- 36 Implementation Roadmap and Definition of Done

APPENDICES

- A. Open Decisions and Architecture Decision Records
- B. Extended API Examples
- C. Death Case State Transition Matrix
- D. Security Risk Register

- E. Role-Permission Matrix
- F. Detailed Test Catalogue
- G. Environment Variables and Secrets Inventory
- H. Requirements Traceability Matrix
- I. Glossary and Data Handling Classes
- J. Source References and Review Checklist

Navigation tip The Word document uses semantic Heading 1/2/3 styles so the Navigation pane can be used to jump between chapters. The contents list is a structured chapter index; page numbers may change as the TDD is tailored.

CHAPTER 01

Executive Summary

Product intent, principal architecture decisions and non-negotiable safety boundaries

1.1 Product vision

Portfolio Immortalized is a secure digital legacy custody platform. It allows a person to store wills, legal records, personal instructions, photographs, audio recordings and videos, nominate trustees and beneficiaries, define controlled-release instructions, and arrange a documented process through which specified materials may be accessed after the person's verified death.

The platform is developed and operated by Pie Corp Limited. The principal product is the mobile application for Android and iOS, supported by a responsive web application. The web platform also provides dedicated trustee, beneficiary and administration experiences. All clients use a shared, server-authoritative security and workflow layer.

The product is not itself a substitute for an attorney, an official registry, a probate court, or a legally valid execution process for a will. It stores and releases materials according to the platform's terms, user instructions, evidence, policy and applicable legal requirements. The interface must not promise that storage guarantees legal validity, probate acceptance, inheritance rights or a particular legal outcome.

1.2 Core outcomes

- Provide a trustworthy, mobile-first vault for important legal and personal legacy material.
- Let users create accounts with minimal friction, then progressively complete profiles and legacy plans.
- Support secure upload, camera scanning, video capture, audio recording, versioning and per-beneficiary assignment.
- Enable trustees to accept invitations, perform designated checks and submit death evidence.
- Use inactivity as an escalation signal only; never infer death solely from the absence of app activity.
- Give administrators end-to-end case oversight, immediate release-hold controls and audited manual intervention tools.
- Provide carefully restricted AI assistance without giving models authority over account access, death status or release decisions.
- Operate with tested restore procedures, strong authentication, least privilege, documented privacy decisions and independent security testing.

1.3 Architectural decisions recommended for the baseline

ID	Decision	Rationale
ADR-001	Next.js web + React Native/Expo mobile, TypeScript throughout	Supports mobile-first native capabilities and a consistent language across clients and server code.
ADR-002	Next.js server-side API/data-access layer for the first release	Reduces service sprawl and centralizes authorization. Separate worker process may be deployed for long-running jobs.
ADR-003	PostgreSQL on Supabase accessed through Prisma from trusted server code	Relational integrity supports relationships, audit trails and case workflows.
ADR-004	Better Auth for identity/session primitives; authorization remains application-owned	Authentication does not replace per-resource authorization or release workflow checks.
ADR-005	Private encrypted object storage for sensitive assets; Cloudinary only where appropriate	Avoids depending on obscurity or expiring URLs as the only confidentiality control.
ADR-006	No automatic death classification and no AI release authority	False-positive release could be irreversible and harmful.
ADR-007	Two-person authorization for final release; immediate one-person emergency hold	Asymmetric control: stopping disclosure is easier than authorizing it.
ADR-008	Provider-neutral AI Gateway; send minimum necessary data	Permits model comparison and privacy controls without coupling product logic to one vendor.
ADR-009	Named admin identities and granular permissions; no shared super-admin login	Makes each privileged operation attributable and reviewable.

ID	Decision	Rationale
ADR-010	Do not introduce blockchain in MVP without a precise threat-model benefit	Hashes and protected append-only logs generally provide simpler integrity evidence; a blockchain would not prove death or legal validity.

1.4 Critical safety invariants

- No time-based job, AI model, trustee alone, or front-end action can directly authorize asset decryption or release.
- An open death case does not itself grant beneficiaries access.
- Every release grant is bound to a case, an asset set, a beneficiary, an authorization record and a time.
- An authorized hold blocks newly created release grants and triggers revocation or invalidation of existing grants as technically possible.
- Critical account, trustee, beneficiary, key and release operations require server-side permission checks independent of interface visibility.
- If evidence, case state or policy evaluation is incomplete or inconsistent, the system fails closed and routes the case to human review.

Security gate Cryptographic escrow and post-death recoverability are architectural risks, not coding details. Before implementing production encryption, a security architect must approve the threat model, key custody, recovery path, HSM/KMS choice, revocation mechanics and disaster-recovery procedures.

CHAPTER 02

Product Scope and Assumptions

Define what the first release does, what it does not do and which questions must remain explicit

2.1 In-scope capability

- Account creation, email verification, sign-in, session management, account recovery and device management.
- Progressive profile completion and consent records.
- Trustee and beneficiary invitations, acceptance, verification status and per-asset access assignment.
- Private vault for PDFs, common image formats, scanned multi-page documents, video, audio and selected supporting files.
- Mobile camera capture, scan clean-up, native file picking, resumable upload, upload recovery and status indicators.
- Inactivity rules, escalating notices, case creation, trustee evidence submission, admin verification, holds and release workflow.
- Administrator user search, account suspension/closure/deletion workflow, relationship management, case queues, manual trigger, case hold, approval/rejection and audit access.
- AI assistance for approved support content, document classification, OCR/scan quality, case summaries and review checklist drafting under explicit controls.
- Operations dashboards, alerts, notifications, backups, monitoring, audit retention and incident handling.

2.2 Explicitly out of scope for baseline unless separately approved

- AI automatically deciding that a user is dead or automatically unlocking files.
- Public blockchain anchoring of every upload. A content hash and independently protected audit trail are the baseline.
- Creation of a legally binding will entirely within the app, electronic witnessing, remote notarization or probate submission automation.
- Automatic identity proofing against government databases unless a lawful provider and jurisdiction-specific method are selected.
- Automatic facial recognition or biometric matching against identity documents.
- Public social features, peer-to-peer content sharing, family account collaboration beyond defined trustee/beneficiary roles.
- AI training on user documents or videos; this is prohibited by the product design unless separately consented, lawful and approved, and is not an MVP feature.

2.3 Assumptions

ID	Assumption	Impact if false
AS-01	The first operating jurisdiction is Kenya, but users may have foreign residence or legal documents.	Jurisdiction matrix and legal advice must be expanded before accepting other jurisdictions.
AS-02	Email via Brevo is the initial notification channel; SMS and push notifications may be added through approved providers.	Inactivity workflow must be adapted if delivery channels are not available.
AS-03	A user may nominate multiple trustees and beneficiaries; policy requires at least two independent trustees for high-risk death workflows, unless approved exception.	Data model and case rules must support policy-controlled quorum.
AS-04	There is a single initial Super Admin account authority, but individual named admin accounts are used in operation.	Initial bootstrap and account recovery require a controlled procedure.
AS-05	Private assets need controlled post-death release; therefore the system requires a recoverable escrow design rather than user-exclusive encryption keys.	If product owner requires strict end-to-end encryption with no service-side recovery capability, post-death delivery design must be reworked.
AS-06	PostgreSQL and object storage are accessed by server-side code; direct mobile access is limited to narrowly scoped pre-signed upload routes where justified.	A redesign is required if client-side direct uploads become the default.

ID	Assumption	Impact if false
AS-07	AI processing of sensitive material is opt-in or tied to a clearly disclosed, necessary service purpose; lower-risk AI use is launched first.	Privacy assessment and AI provider choices must change.

2.4 Terminology distinctions

- "Owner" means the living account holder who created and controls the legacy plan.
- "Death report" means a claim or notification that the owner may have died. It is not a verified determination.
- "Death case" means a controlled investigation record with evidence, status, reviewers and decisions.
- "Release authorization" means an approved, auditable decision permitting a defined release grant.
- "Release grant" means narrowly scoped access for one beneficiary to a defined group of assets during an authorized period.
- "Verified activity" means meaningful, authenticated owner activity accepted under the configured inactivity policy; delivery receipts, background refreshes and analytics pings do not count.
- "AI suggestion" means non-authoritative output that requires application validation and, for material case decisions, human review.

2.5 Open decisions before implementation freeze

Decision	Owner	Required by	Recommended default
Key custody provider and post-death recovery policy	Product owner + security architect	Before encryption implementation	Envelope encryption with KMS/HSM-protected KEKs and a release service requiring independent approvals.
Jurisdictions supported at launch	Product owner + legal adviser	Before public beta	Kenya-only supported policy; others flagged as unsupported or requiring legal review.
Evidence acceptance rules and required case quorum	Legal adviser + operations	Before death-case integration tests	Two-person independent review with a documented evidence checklist; exceptions require recorded reasoning.
Inactivity durations and reminder cadence	Product owner + legal adviser	Before production notices	Configurable policy; initial proposal 90 days plus escalation/grace periods, not a claim of death.
Storage provider/region and backup geography	Security + operations	Before production data	Private bucket and approved region; validate cross-border transfer impact.
Admin role holders and separation-of-duties mapping	Company leadership	Before launch	No shared accounts; release requester and final approver must be distinct named users.
AI vendor/service tier and allowed data classes	Privacy lead + security	Before AI pilot	Paid/commercial API subject to DP assessment; no raw legal documents sent until approved.

CHAPTER 03

Personas, Roles and Trust Boundaries

Actors and the boundaries that separate their permissions

3.1 Personas

Persona	Primary goal	Typical access
Owner / account holder	Store private materials and define legacy instructions	Own profile, own vault, trustees, beneficiaries, policies, sessions and consent.
Trustee	Support a controlled verification workflow	Only assigned owner relationships and case tasks; cannot browse the owner's vault by default.
Beneficiary	Receive specifically assigned materials after authorization	Own identity checks, assigned release grants and delivered files only.
Support Admin	Resolve ordinary account problems	Limited metadata, authorized account maintenance and support cases; no general file decryption.
Security Admin	Protect platform and investigate suspicious activity	Security event metadata, session revocation, emergency holds and incident handling.
Case Reviewer	Review death evidence and prepare a recommendation	Death case, submitted evidence and permitted identity metadata; no unilateral final release.
Release Approver	Independently approve or reject release authorization	Case review data and final approval workflow; must not be the release requester.
Super Admin	Manage privileged configuration and admin role assignments	System-level configuration, role provisioning and emergency operations with extra controls.
Auditor	Verify adherence to policy	Read-only, scoped logs and reports; cannot change cases or access decrypted content by default.
Operations Engineer	Keep service available	Infrastructure metrics and deployment access; production data access is exceptional and audited.

3.2 Trust boundaries

- Boundary A - untrusted clients: mobile app, browser, deep links, uploaded files, user-generated text and all client-supplied metadata.
- Boundary B - authenticated API: verifies session and requested actor, validates inputs and derives identity from the trusted session rather than the request body.
- Boundary C - authorization/data access layer: validates object-level permissions, owner relationship, state machine preconditions and admin scopes.
- Boundary D - database: enforces constraints, transactions and RLS where exposed; privileged database credentials never reach clients.
- Boundary E - object storage: stores ciphertext for high-confidentiality assets; object path is not a permission.
- Boundary F - key-management/release service: separate access boundary for key unwrap/decryption authorization; no browser-facing key API.
- Boundary G - external providers: Brevo, AI providers, push/SMS vendors, telemetry and optional Cloudinary. Each receives only the minimum required data.
- Boundary H - privileged staff: named accounts, MFA/passkeys, role-based permissions, step-up challenges and independent approval for release.

3.3 Abuse cases to assume

- A trustee attempts to falsely report the owner as deceased to obtain access.
- A beneficiary attempts to alter the URL or identifier and access another beneficiary's materials.
- A compromised owner phone remains authenticated and is used to change trustees or recovery channels.
- A staff account is phished, then used to view private metadata or alter a death case.
- A malicious document includes instructions designed to manipulate an AI model processing it.
- A file upload uses a spoofed extension, polyglot payload, decompression bomb or malicious media codec.
- A background job executes twice or out of order and produces duplicate notifications or release grants.
- A compromised email account receives an invitation or reset link, so email alone cannot authorize a final release.
- A scheduled inactivity job marks an owner as suspicious while the owner is hospitalized, travelling or unable to use the app.
- A public/signed media URL is copied and used outside the application.

CHAPTER 04

Functional Requirements and Acceptance Criteria

Traceable requirements for product behaviour

4.1 Requirement identifiers and priority

Requirements use stable IDs such as AUTH-001, VAULT-003, CASE-012 and ADM-008. Each requirement should be mapped to code modules, database entities, API endpoints, tests and the user-facing screens it affects. Priorities are P0 (launch blocker), P1 (launch target) and P2 (post-launch candidate).

Domain	Requirement	Priority	Acceptance summary
AUTH-001	User can sign up with minimal required fields and verify email.	P0	Unverified accounts cannot access protected vault functions; repeated verification is rate-limited.
AUTH-002	User can recover account without relying solely on one device or email token.	P0	Recovery includes risk checks, notifications, session revocation and secure fallback.
AUTH-003	Mobile biometric unlock is local and optional.	P1	Biometric material is not sent to the server; server APIs still validate sessions and authorization.
PROF-001	Additional profile details can be completed after signup.	P0	Incomplete profile does not prevent basic sign-in; gated features display clear requirements.
VAULT-001	Owner can upload PDF, image, audio and video files.	P0	Type/size checks, encryption, integrity confirmation and durable object registration complete before success.
VAULT-002	Owner can capture multi-page document scans in app.	P1	Page crop, rotate, reorder, retake and preview before final save.
VAULT-003	Owner can record video/audio or select an existing recording.	P1	User sees upload and encryption state; interrupted upload can resume or safely restart.
PEOPLE-001	Owner can invite and manage trustees and beneficiaries.	P0	Invitation tokens are scoped, expiring, single use and acceptance is audited.
POL-001	Owner configures legacy instructions and item-to-beneficiary assignment.	P0	Preview shows exactly which assets and recipients are affected before save.
MON-001	System evaluates inactivity in scheduled jobs.	P0	Meaningful verified activity resets the clock; system-generated activity does not.
CASE-001	Suspected death creates an investigation, not an automatic release.	P0	No release grant exists solely because inactivity threshold was reached.
CASE-002	Trustees can submit evidence and required co-signature.	P0	Identity, assigned relationship, quorum, deadlines and evidence hashes are recorded.
CASE-003	Admin can open, hold, reopen, reject or route cases.	P0	Every transition is authorized, validated and appended to audit records.
REL-001	Final release requires independent authorization.	P0	Requester and final approver differ; state, evidence, hold and policy checks pass.
ADM-001	Admin can search accounts and inspect permitted metadata/activity.	P0	Search is permission-gated, audited and does not expose decrypted contents by default.
ADM-002	Admin can suspend/close/delete under a controlled process.	P0	Legal holds, case preservation, soft-delete and purge rules enforced.
AI-001	AI is routed through policy-aware gateway.	P0 for any AI feature	No provider API key on client; data class and consent verified before submission.
AI-002	AI cannot authorize death or release.	P0	No callable tool or service endpoint gives the model this authority.
OPS-001	Security events and critical workflow outcomes are audited.	P0	Records include actor, target, action, result, timestamp, request/case correlation and integrity protection.

Domain	Requirement	Priority	Acceptance summary
DR-001	Backups can be restored into a tested isolated environment.	P0	Documented restore test demonstrates usable database and asset/key consistency.

4.2 Cross-cutting acceptance rules

- All protected API endpoints return a consistent error envelope and avoid leaking sensitive internal details.
- Every resource request validates object-level authorization, not merely a top-level role.
- All multi-step operations are safe under retry; duplicate requests do not duplicate money-like effects, notices or grants.
- Critical state transitions occur transactionally with an audit record/outbox event, or use an explicit recoverable saga when spanning systems.
- No sensitive content appears in analytics, crash-reporting tools, push previews or ordinary application logs.
- All core screens implement loading, empty, success, validation-error, permission-denied, offline and retry states.
- Destructive or high-risk actions show the exact target and consequence, require fresh authentication and are auditable.

CHAPTER 05

Non-Functional Requirements and Quality Attributes

Measurable target quality; proposed targets must be validated against hosting and cost

Attribute	Target / policy	Verification
Confidentiality	Private files are encrypted before persistence in object storage; keys are separately controlled.	Threat model, code review, key-management test and object-store inspection.
Authorization	Every endpoint checks actor, resource relationship, state and scope server-side.	Negative integration tests and penetration testing.
Availability	Proposed MVP service objective 99.5% monthly, excluding documented maintenance; revise after provider selection.	Synthetic checks and monthly SLO report.
Performance	P95 authenticated metadata API target < 500 ms under defined load, excluding AI, upload and external-provider latency.	Load test with documented dataset and concurrency.
Upload reliability	Uploads resume after interruption where the chosen storage supports it; otherwise fail without corrupting committed metadata.	Network interruption and checksum tests.
Mobile startup	Proposed target: useful shell quickly on supported mid-range devices; target measured on a documented device matrix.	Cold/warm startup profiling on Android and iOS.
Scalability	Stateless web/API layer can scale horizontally; async jobs use bounded concurrency.	Load and queue saturation tests.
Recoverability	Initial proposed objectives: database RPO <= 24 hours and service RTO <= 8 hours; stricter targets require cost assessment.	Scheduled restore drills and evidence report.
Auditability	All privileged and case-state-changing actions have actor attribution and an event trail.	Automated tests and quarterly audit sampling.
Accessibility	Meet WCAG 2.2 AA for web target where applicable; mobile controls support platform accessibility labels and dynamic text.	Automated scan + manual screen-reader review.
Privacy	Data minimization and retention schedule for every field and provider; no hidden background death inference.	DPIA, data inventory and retention tests.
Maintainability	Typed boundaries, linting, CI tests, code ownership, ADRs and reviewed database migrations.	CI gates and periodic architecture review.
Portability	Avoid provider-specific identifiers throughout core business model where abstraction is practical.	Architecture tests and documented migration plan.
Observability	Structured, redacted logs, service metrics, traces and actionable critical alerts.	Alert simulation and incident runbook exercises.

5.1 Mobile network assumptions

Mobile clients will often operate on unstable or metered networks. The application must handle app suspension, low memory, offline transitions, expired sessions and background upload restrictions. A pending upload must be represented with an explicit status and a safe retry action. The application must not mark a document as stored until the backend confirms object persistence, integrity verification and metadata commit.

5.2 Availability boundaries

External AI, email, push and object-storage dependencies can fail independently. The core vault browsing experience should remain usable when AI is unavailable. Email delay must never be interpreted as owner inactivity evidence. Failure to contact a trustee must not advance a case into release. Provider outages should leave workflow state durable and retryable.

CHAPTER 06

Information Architecture and Navigation

Primary navigation and cross-surface conventions

6.1 Mobile primary navigation

Destination	Purpose	Primary views
Home	Reassurance and required next actions	Vault security status, recent activity, setup checklist, alerts and upload activity.
Vault	Private legacy asset management	Documents, IDs, wills, photos, videos, audio, folders, file detail and upload actions.
People	Relationship and recipient management	Trustees, beneficiaries, invitation status, relationship forms and access assignments.
Legacy	What should be released and to whom	Instructions, recipient-specific messages, asset assignment summary and release policy explanation.
Settings	Identity, security and preferences	Profile, devices, login methods, app lock, recovery, notification preferences, privacy, support and account closure.

6.2 Web navigation

The owner web application mirrors the mobile domain model but may use a side navigation rail and responsive two-column layouts on desktop. Trustee, beneficiary and administrator portals are separate route namespaces and layouts. Route isolation is not a security boundary; the server must enforce role and relationship checks on every request.

6.3 Interaction principles

- Explain security states in plain language. “Investigation opened” must not be phrased as “death confirmed.”
- Use explicit status chips with textual labels and icons; never rely on color alone.
- Show progress and resumability for scanning, encryption, uploads and long-running requests.
- Require confirmation before irreversible changes and display the number of affected items/people.
- Use accessible labels for camera, upload, record, play, retake, pause, save and remove controls.
- Hide content previews from OS notifications by default; offer a privacy-conscious setting.
- Provide localized date/time formatting while storing machine timestamps in UTC. Jurisdiction-specific dates should be explicit.
- Prevent accidental disclosure through screen previews: use app-switcher privacy treatment where feasible and recommend device-level security.

6.4 Shared component library

- Buttons: primary, secondary, destructive, quiet and loading; destructive action style includes target text.
- Form elements: labels, helper text, inline validation, accessible error summary and required/optional state.
- Status components: invitation pending, verified, unreachable, suspended, investigation open, on hold, review required, release approved.
- File components: type icon, name, date, confidentiality badge, integrity state, upload progress and access policy summary.
- Case components: state, severity, owner/respondent, evidence checklist, reviewer assignment, due date and audit timeline.
- Security banners: new device, recovery changed, case opened, administrative change, hold active and suspicious activity.

CHAPTER 07

Mobile App Screen Specification

Detailed screen inventory and expected states for Android and iOS

7.1 Onboarding and authentication screens

Screen	Fields/actions	Validation and security behaviour
Welcome	Create account; Sign in; Terms; Privacy	No sensitive document content shown before authentication.
Sign up	Name/display name, email, chosen login method, agreement consent	Normalize email; rate limit; verify email; terms version recorded with timestamp.
Email verification	Code or verified link, resend action	Expiring one-time token; throttled retries; generic error to avoid account enumeration where appropriate.
Sign in	Email + configured credential/passkey/OTP method	Risk checks; rate limit; session is server-issued; suspicious attempts logged.
MFA / step-up	Passkey or secondary factor where configured	Use for sensitive operations; challenge bound to action and expires quickly.
App lock setup	Biometric capability prompt; optional local PIN	Biometric prompt delegates to OS; no fingerprint/face template stored by app. Local PIN is not backend-only identity proof.
Account recovery	Recovery route, verification steps, device/session revocation	Avoid weak security questions as the only factor; notify established channels after recovery.
Device sessions	Current device and sessions; revoke actions	Revocation takes effect server-side; client cache cleared when possible.

7.2 Home dashboard

- Show vault security status using neutral language: “Protected”, “Setup incomplete”, “Security action needed” or “Investigation underway”.
- Offer shortcuts to upload a document, scan pages, record a video, manage people and review legacy instructions.
- Show recent account activity and upload states, not content details in push previews.
- Show profile/legacy plan readiness: profile, at least required trustee quorum, beneficiaries, recovery method and asset assignments.
- Show user-facing notices if an inactivity investigation is open, including an explanation and safe route to contact support. The notice must not reveal confidential trustee allegations unnecessarily.
- Never display a false assurance such as “we know you are alive” or “death automatically detected”.

7.3 Vault screens

Screen	Required behaviour
Vault overview	Filter by Documents, Wills, IDs, Photos, Videos, Audio and Other; search metadata only; empty states guide upload.
Add material sheet	Offer Select file, Scan pages, Record video, Record audio, Choose video/photo and Write a message.
Document scanner	Permission explanation; multi-page capture; crop; rotate; reorder; retake; quality warning; save as PDF or images.
File picker	Restrict allowed file types and size; display selected name/size; do not trust client extension.

Screen	Required behaviour
Upload review	Edit title, description, category, sensitivity, version intent, recipients, release conditions and AI assistance consent.
Upload progress	Separate local preparation/encryption from transfer/storage verification; support retry/resume where possible.
Asset details	Metadata, version list, integrity status, assigned recipients, release policy, access history and replace/retire actions.
Secure preview	Short-lived authorized stream/download after checking policy; warn that screen capture cannot be fully prevented on all devices.
Version history	Show original/version timestamps and which version is designated; avoid silent overwrites.
Delete/retire asset	Confirm asset and dependent assignments; apply retention/legal hold rules; record deletion request and completion state.

7.4 Recording experience

For video/audio capture, the app requests camera/microphone permissions only after the user selects the feature and receives a plain-language reason. Before recording, show the intended privacy and recipient setting. After recording, show preview, playback, retake, title and upload controls. The app should never start recording in the background. Interrupted recording and upload should produce an explicit recover, discard or resume choice. Metadata such as GPS location should be stripped unless essential and specifically enabled by the user.

7.5 People screens

Screen	Required behaviour
People overview	Separate Trustees and Beneficiaries, show invitation/verification status and number of assigned assets.
Add trustee	Name, email, phone, relationship, optional role or legal representative information; explain responsibility and data use.
Invite trustee	Review invitation message, consent/privacy notice and scope; send single-use expiring link.
Trustee detail	Contact, accepted status, verification state, case tasks and removal/replacement workflow.
Add beneficiary	Minimum identity/contact details needed; relationship optional where not required; avoid collecting ID until necessary.
Assignment editor	Choose particular files/messages per recipient; preview exact access before save.
Change relationship	Fresh authentication, summary of the change, owner notification and audit record.
Trustee case response	Show assigned case, steps, evidence requirements, safe upload and confirmation. Never show more of owner's vault than needed.

7.6 Legacy, profile and settings screens

- Profile: legal identity fields progressively collected according to jurisdiction and service need; profile completeness clearly marked.
- Legacy plan: recipient map, item-by-item assignments, message instructions, review summary, last modified and current status.
- Release settings: explain inactivity escalation and what it does and does not mean; avoid suggesting the app can automatically verify death.
- Recovery: enrolled factors, recovery contact verification, download/use recovery codes if supported, and recovery readiness check.

- Notifications: channel preferences for ordinary updates, while mandatory security/death-case alerts may require a separately disclosed policy.
- Privacy: AI processing opt-ins and history, data export request, privacy contact, consent version and account closure request.
- Help/support: FAQs, case status, authenticated contact route and emergency security/report form.
- Account closure: impact preview, confirm identity, explain legal hold/retention exceptions and provide request status.

7.7 Mobile-specific technical requirements

- Use native secure storage for refresh/session artifacts as supported by the auth integration; avoid storing raw encryption keys in general app storage.
- Use iOS Keychain/Secure Enclave and Android Keystore capabilities through audited platform APIs where applicable.
- Support universal/app links with domain association and token validation; reject untrusted deep-link origins.
- Apply certificate-validation defaults; do not introduce custom certificate pinning without a rotation and recovery plan.
- Disable sensitive content in screen previews where practical, while explaining that no client can guarantee prevention of screenshots or external camera capture.
- Protect local caches. If offline previews are permitted, encrypt them locally, define expiry, revoke local access on logout and test backup behavior.
- Provide graceful permission-denied handling for camera, mic, biometric and notification permission changes.
- Use crash-reporting redaction rules; app crash reports must not contain document text, signed URLs, tokens or encryption material.

CHAPTER 08

Web Application and Portal Specification

Owner web, trustee, beneficiary and administrative surfaces

8.1 Owner web portal

The owner web portal should have feature parity for account management, legacy planning, vault administration and viewing authorized assets, subject to secure browser handling. It should not simply wrap mobile screens at desktop width. Desktop can use a responsive sidebar, dense-but-readable tables, drag/drop upload and batch metadata editing. High-risk security operations still require step-up authentication.

8.2 Trustee portal

- Invitation acceptance: show requesting owner identity, scope of trustee responsibility, privacy notice and explicit accept/decline action.
- Dashboard: only relationships and cases assigned to the authenticated trustee.
- Case evidence submission: evidence type, issue date, issuing authority/country, notes, upload status and declaration of accuracy.
- Co-sign workflow: show what the trustee is confirming, evidence summary and consequences; require fresh authentication.
- Report status: pending, more information requested, held, approved or rejected; do not disclose other trustees' private data beyond the case need.
- Conflict reporting: allow a trustee to report uncertainty, a conflict of interest or suspected abuse, with mandatory routing to review.

8.3 Beneficiary portal

- Access invitation must not disclose asset contents before identity verification and release authorization.
- Verify email/phone or stronger identity proof according to release-risk policy; delivery channel alone does not prove legal entitlement.
- Show the exact authorized items and permitted actions after a release grant is validated.
- Provide a recipient-specific release notice and acknowledgement of sensitive material.
- Secure viewing and downloading should use grant-scoped, time-limited authorization; avoid permanent public links.
- Log grant creation, first access, download attempts, expiration and revocation without logging file content.

8.4 Admin web route groups

Route group (illustrative)	Contents	Permission examples
/admin/overview	KPIs, critical alerts, release queue, service health	Admin dashboard access.
/admin/users	Search/filter user directory and account detail	User-read scope; restricted fields minimized.
/admin/users/:id/activity	Timeline, sessions, security flags, relationship map	User audit scope; every view logged for sensitive fields.
/admin/relationships	Trustee/beneficiary correction and invitations	Relationship-manage scope plus step-up for material changes.
/admin/cases	Death case queue, evidence checklist, assignment and SLA	Case-review scope.
/admin/cases/:id/decision	Approve/reject/reopen/hold, recommendation and decision	Case reviewer or release approver; independent separation.
/admin/releases	Authorized release grants, failed grants, revocations	Release-approver scope; no raw key access.
/admin/security	Security alerts, sessions, login anomalies, key service events	Security-admin scope.
/admin/audit	Append-only event search and controlled exports	Auditor or higher-scoped role.

Route group (illustrative)	Contents	Permission examples
/admin/operations	Storage, queues, email provider, backups and system health	Operations scope; no default private-file access.
/admin/ai-review	AI processing logs, quality reports, disputed suggestions	AI reviewer access; prompts/content redacted by default.
/admin/configuration	Notification templates, policy version, feature flags	Super-admin; changes versioned and peer-reviewed.

8.5 Accessibility and layout details

All portals must provide clear focus states, keyboard navigation, accessible table labels, error summaries, screen-reader announcements for state changes and adequate contrast. Admin tables should support pagination rather than unbounded result lists. Critical decision pages should place the case state, evidence requirements, owner alert status, hold status and independent approval status above action controls. Destructive or release actions should not sit adjacent to routine navigation without visual separation.

CHAPTER 09

Admin Console Specification

Privileged operation, account controls, monitoring and accountable intervention

9.1 Admin dashboard widgets

Widget	Data shown	Controls / alerts
Platform health	Accounts by status, current active users, upload success, worker backlog	Operational incident links; no private asset contents.
Death case queue	New, overdue, awaiting evidence, pending second review, held, ready for final decision	Acknowledge, assign, escalate, place hold.
Critical security alerts	Compromised session signals, repeated recovery failures, unusual privileged activity	Revoke session, impose temporary hold, open incident.
Account lifecycle	Registration funnel, unverified accounts, inactive records, closure requests	Verification reminder, policy-compliant closure workflow.
Trustee/beneficiary health	Pending invites, failed delivery, unreachable contacts, identity-check backlog	Resend/revoke invitation, request correction.
Storage and encryption	Storage capacity, failed uploads, encryption failures, key service status	Investigate; do not expose plaintext key material.
Communications	Brevo delivery/retry status, push/SMS failure if integrated	Retry safe messages; avoid bulk sends without approval.
AI quality and governance	Task volume, error rate, provider failures, redacted samples/ratings	Disable provider or feature, open review; no model-generated decision control.

9.2 User directory and detailed profile

Search should support stable account ID, normalized email, carefully scoped phone lookup and approved status filters. Searches of sensitive attributes should themselves be auditable. Account detail should present tabs: Overview, Profile, Relationships, Activity, Security, Vault metadata, Cases, Consents, Support and Audit. A support administrator may see file names only if that is necessary for the assigned support task; decrypted content access is separate and exceptional.

9.3 Account restrictions and deletion

- Suspend: block new sign-in or protected actions according to incident policy while preserving evidence and records.
- Revoke sessions: invalidate server-side sessions, including other devices. Client cleanup is best effort and is not the control boundary.
- Restrict asset access: freeze selected operations or place related release grants on hold without deleting objects.
- Close account: disable routine use after the request and validation; preserve records subject to legal obligations and declared retention.
- Queue deletion: calculate dependent data, legal holds, open cases, backups and related beneficiaries; display a dry-run summary before execution.
- Hard purge: run a controlled asynchronous job only after eligibility checks; record which categories were purged, retained or cryptographically erased.
- Never implement a generic cascading delete from User to DeathCase, AuditEvent, evidence and ReleaseGrant. Referential actions must preserve accountability and legal hold.

9.4 Relationship administration

Admins may correct or remove a trustee/beneficiary when authorized for support, safety, deceased/trustee incapacitation, dispute or legal-process reasons. Before applying a relationship change, the service evaluates whether the user is alive and able to consent, whether a death case is open, whether a release is pending, and whether a legal hold exists. Notify the owner when safe and appropriate; do not notify a

suspected attacker when doing so would compromise an active investigation. Require the actor to record reason code and free-text rationale.

9.5 Manual case actions

Action	Allowed actor	Mandatory controls
Open death investigation	Case Reviewer, Security Admin, Super Admin under policy	Reason, basis, target account, alert; no death status assertion.
Emergency hold	Security Admin, Case Reviewer, Release Approver, Super Admin	Immediate effect, reason, notification to case queue, later review; cannot be blocked by ordinary workflow.
Request additional evidence	Case Reviewer	Checklist, due date, minimal disclosure, recipient-specific notification.
Recommend approval/rejection	Case Reviewer	Evidence checklist and reasoned recommendation; cannot be final approver for same request.
Final release approval	Release Approver independent from requester/reviewer under policy	Fresh auth, independent second approval, hold and veto check, asset scope confirmation.
Manual release initiation	Case Reviewer or authorized admin	Starts full review pipeline; never bypasses state machine or evidence requirement.
Reopen or dispute case	Case Reviewer / Security Admin	Reason, preserved prior decisions, immediate release suspension if material conflict.
Reject and close	Authorized case role	Reason code, evidence outcome, appeal/escalation route and owner/trustee notices as appropriate.

9.6 Administrator separation of duties

- No shared administrator identities. Named individual accounts only.
- MFA or passkeys are mandatory for administrators. Step-up auth is required for all sensitive changes.
- Final release approver cannot be the case requester, evidence submitter, primary reviewer or another approving identity prohibited by policy.
- Configuration changes affecting quorum, hold periods, evidence rules, retention or AI processing require peer review and immutable change history.
- Break-glass access is short lived, case-bound, reasoned and alerted; do not include a general “decrypt everything” interface.
- Administrator privileges must be reviewed at onboarding, role change, offboarding and a recurring schedule.

CHAPTER 10

Core User Workflows

Sequence-level behaviours, exception handling and audit points

10.1 Registration and progressive completion

- User accepts current terms and privacy information. Store versioned consent events separately from mutable profile values.
- User creates account using minimal required data and a configured login method.
- Application verifies email before enabling protected operations. Phone verification is introduced only when required for a chosen feature or assurance level.
- User enters the dashboard and sees a checklist; additional profile, recovery and jurisdiction details can be completed later.
- When user invites trustees, each invite token is generated server-side, single-use, purpose-bound, expires and is stored hashed where applicable.
- Trustee invitation acceptance binds the authenticated person to the invitation only after validating address/channel, expiry, intended owner, assigned role and duplicate state.
- User defines beneficiaries and assigns assets/messages. A preview screen states precisely what each beneficiary is intended to receive.
- All meaningful changes create versioned records and audit events; accepted legacy plan version is identifiable.

10.2 Uploading a signed will

Step	Client action	Server / worker action	Success condition
1	Choose PDF, image or scan route	Create upload intent; authenticate user; validate quota and permitted type	Authorized upload intent returned.
2	Capture/select files and review them	Apply encryption policy and safe media-processing limits; generate upload session	Client knows preparation and upload status.
3	Encrypt and upload chunks	Store ciphertext privately; record checksums and provider object reference	Transfer finishes or can safely resume.
4	Confirm upload	Verify object exists, size, checksum, encryption/key envelope and scan status	Metadata and upload state commit idempotently.
5	Review metadata/recipients	Validate owner permission, beneficiary assignments and legacy plan version	Asset linked to user and approved configuration.
6	Finalize	Write asset version and audit event; emit notification event through outbox	UI displays verified successful save.

10.3 Replace or update a will

The default operation is version creation, not overwriting. When the owner uploads a replacement, the interface asks whether this is a new version of an existing asset or a separate document. The owner previews the current and new version metadata and confirms which version should be released. The system preserves a tombstone and version metadata for retired versions, respecting retention policy. If a death case is already open, an asset replacement may be blocked or require a policy-specific elevated confirmation; active case evidence must not silently change under reviewers.

10.4 Trustee invitation lifecycle

- Owner creates invitation and confirms identity/contact details.
- Backend creates invitation record with random token, hash, expiry, role, owner ID and intended recipient channel.
- Brevo delivers a neutral invitation message that avoids exposing sensitive information.

- Recipient opens the application/site, authenticates or creates an account, verifies control of the intended channel and reviews responsibility disclosure.
- Recipient accepts or declines; server atomically consumes the invitation token and records outcome.
- If invitation expires, owner may resend; prior token becomes invalid. Contact changes require revocation of outstanding invitation.
- Acceptance does not grant access to wills; it grants only the trustee role and assigned workflow permissions.

10.5 Beneficiary release experience

- Beneficiary receives a generic notice that a secure legacy item may be available; do not attach a document or expose title unless policy permits.
- Beneficiary opens the verified official app/domain and signs in or enrolls.
- Release service verifies the specific grant, identity assurance, case state, hold status, grant expiry and any extra factor.
- Beneficiary sees exact authorized item list and recipient instructions; every fetch is checked server-side.
- If downloading is permitted, system emits short-lived authorization; if in-browser viewing is provided, ciphertext remains protected at rest and plaintext is delivered only after a valid decision.
- Beneficiary acknowledgement and access event are recorded. Watermark any derivative where legally and technically appropriate, without altering the archived original.

10.6 Account compromise

- Owner or monitoring rules report a suspected compromise.
- Security workflow can revoke sessions, disable a compromised credential, freeze sensitive changes and place pending death release on hold.
- Owner identity is re-established through recovery with strengthened checks and notices to pre-existing trusted channels where safe.
- Review recent changes to trustee/beneficiary, email, phone, recovery and legacy assignments. Mark suspicious changes for owner confirmation.
- Restore only verified settings; preserve the pre-incident state and event history.
- Close the incident only after documenting impact assessment, affected data, notifications and required regulator/legal escalation.

CHAPTER 11

Inactivity, Death Verification and Release State Machine

The core safety workflow; inactivity is a concern signal, never proof of death

11.1 Meaningful activity policy

The system records "last meaningful owner activity" from a limited set of events that demonstrate recent authenticated interaction. Candidates include successful fresh authentication, explicit secure settings confirmation, or a device-bound operation signed after local biometric/passkey verification. Product and security owners must finalize the accepted event list. App launches, notification receipts, analytics pings, background token refreshes, third-party email opens and a device merely being online do not count. A biometric success at the OS level only establishes local user-presence for that action; server records must validate the associated challenge and session.

11.2 Suggested initial timing policy

Stage	Illustrative default	System behaviour
Active	Before configured inactivity threshold	No death implication; display current policy and last meaningful activity.
Reminder	Around 75 days since activity for a proposed 90-day threshold	Send discreet reminder through verified channels; allow policy-configured cadence.
Threshold reached	At 90 days, subject to owner-approved policy	Open inactivity concern record; notify owner and issue first escalation. Do not mark deceased.
Grace period	Proposed additional 14 days	Attempt approved channels; record delivery failures; never infer death from delivery/read status.
Trustee review requested	After grace and escalation rules, if no meaningful activity	Notify designated trustees and admin case queue; status becomes investigation-needed.
Evidence review	As evidence is submitted	Track trustee quorum, evidence completeness, owner challenges and admin decisions.
Hold period	Proposed minimum 72 hours after verified critical notice, subject to legal/policy review	No final grant before hold expires. Any credible owner response, conflict or security alert pauses release.
Final decision	After hold and approvals	Two-person independent release authorization; exact asset and recipient scope confirmed.

> **Policy note** The timings above are a proposed baseline for product discussion, not a statement of law and not copied as a final approved requirement. The uploaded blueprint uses inconsistent 48-hour and 72-hour hold references. Select one approved policy and use it consistently in UI, code, notifications and test cases.

11.3 Death case states

State	Entry condition	Permitted exit(s)	Must not happen
NONE	No case exists	INACTIVITY_REVIEW or REPORTED	No death label from passive signals.
INACTIVITY_REVIEW	Configured inactivity rule exceeded	OWNER_RECONFIRMED, TRUSTEE_REVIEW, CLOSED_NO_EVIDENCE	No release grant.
REPORTED	Trustee/admin/external claim submitted	EVIDENCE_PENDING, DISPUTED, REJECTED	No release grant.
EVIDENCE_PENDING	Minimum submission exists but completeness unresolved	UNDER_REVIEW, NEEDS_MORE_INFO, DISPUTED	No automatic approval based on file type or AI extraction.
UNDER_REVIEW	Reviewer assigned and checklist started	NEEDS_MORE_INFO, HOLD, READY_FOR_SECOND_REVIEW, REJECTED	Reviewer cannot final-approve own request.
NEEDS_MORE_INFO	Evidence incomplete	EVIDENCE_PENDING, DISPUTED, REJECTED	Deadlines do not auto-authorize release.
DISPUTED	Owner response, conflicting evidence, suspected fraud or legal dispute	HOLD, UNDER_REVIEW, REJECTED, CLOSED	No release while dispute is unresolved.
HOLD	Emergency or policy hold	UNDER_REVIEW, REJECTED,	Release worker must honor hold

State	Entry condition	Permitted exit(s)	Must not happen
		READY_FOR_SECOND_REVIEW only after resolution	immediately.
READY_FOR_SECOND_REVIEW	Primary review recommends approval; safety conditions satisfied so far	AUTHORIZED or HOLD or REJECTED	No grant before independent approval and re-check.
AUTHORIZED	Required independent approvals recorded and policy re-evaluated	RELEASE_IN_PROGRESS or REVOKED before access	No grant to unassigned recipient/assets.
RELEASE_IN_PROGRESS	Grant issuance jobs initiated	RELEASED, PARTIAL_FAILURE, HOLD	Retry must not broaden scope or duplicate grants.
RELEASED	Required grants issued and delivery state recorded	REVOKED/EXPIRED where supported; case archive	Cannot erase audit trail or undo copies already downloaded.
REJECTED	Decision rejects report/release	REOPENED only with authority and new evidence	No silent deletion or repeat bypass.
CLOSED_NO_EVIDENCE	Investigation closed without adequate evidence	REOPENED under new evidence/policy	No release.
REVOKED/EXPIRED	Grant invalidated or timed out	New grant requires a fresh valid case/policy check	Existing URL alone must not bypass revoked grant.

11.4 Evidence review checklist

- Confirm the submitter is an assigned trustee or otherwise authorized to report under policy.
- Confirm identity attributes match the owner profile sufficiently for the approved process; flag transliteration, amended names and jurisdictions for human examination.
- Check evidence type, issuing authority, date, signs of alteration and provenance; technical image checks are supportive only.
- Record original evidence file hash, upload time, actor, review actions and redacted extracted fields. Preserve the source artifact according to retention policy.
- Look for contradictory information and a credible owner response. A failure to respond is not proof of death.
- Check whether the case has an open dispute, account-compromise incident, recent recovery/change event, court order or legal hold.
- Record reviewer's rationale and evidence relied upon. AI-extracted values must be marked as candidate values until verified by a person.

11.5 Owner response and veto/hold

A verified owner response during any pre-release stage must suspend the case immediately, stop new grant creation, notify the case queue and create a review task. It must not automatically lock the legitimate owner out of the account. If the owner response could itself be compromised, the case remains on hold until identity and account security are resolved. A veto is recorded as a workflow event and does not erase the report or prevent a later report if new independent evidence is submitted.

11.6 Manual initiation by administrator

A manual initiation action means "open an investigation now," not "declare the user dead." The administrator must select a reason, record evidence source and identify the basis for proceeding outside inactivity timing. The workflow then uses the same evidence, review, hold, second-approval and asset-scope controls as any other case. A manual override cannot bypass statutory requirements, legal holds, unresolved disputes or independent authorization.

11.7 Release transaction safety

- At final approval, the system re-reads current case state, active holds, owner response events, trustee quorum, recipient identities, asset versions and policy version in a transaction or equivalent consistency barrier.
- Approval records are immutable; correcting a decision creates a superseding event rather than changing history.
- Release grants are generated only for explicit asset IDs and beneficiary IDs, not an unbounded "all files" rule unless the owner's plan expressly permits it and counsel approves the policy.
- Key release is a distinct operation requiring a release service decision. The general app server must not bypass it.
- Every asynchronous release job is idempotent and checks the latest hold/revocation status before completing each grant.
- If a hold is placed while the release pipeline is running, the pipeline stops before further grants; any credentials already issued are revoked where possible and the case is escalated.
- No system can claw back plaintext already downloaded by a recipient. The UI must explain controlled access does not guarantee that a recipient cannot copy material once legitimately obtained.

CHAPTER 12

File Capture, Upload and Asset Lifecycle

File validation, encryption, progress, versioning, watermarking and secure delivery

12.1 Supported file classes

Class	Proposed accepted formats	Notes
Documents	PDF; JPEG/JPG; PNG; HEIC/HEIF subject to server conversion policy; selected office formats only if approved	Legal docs keep original bytes; previews are derived artifacts. Consider malware scanning and content-disarm for supported formats.
Scans	Image pages and generated PDF	Allow reordering and retaking before commit. Preserve source pages or maintain clear version lineage.
Video	MP4/H.264 baseline; additional formats after tested transcoding matrix	Large upload resumability, safe codec processing and private playback.
Audio	M4A/AAC, MP3, WAV within configured size/duration policy	Transcription is optional, disclosed and separately controlled.
Photos	JPEG/PNG/HEIC and other approved types	Strip EXIF/GPS from derivatives by default; preserve original only when necessary and consented.
Evidence	PDF/image formats accepted by case policy	Separate case-evidence classification, malware scanning and chain-of-custody metadata.

12.2 Upload state machine

State	Description	Allowed transitions
CREATED	Upload intent created; no confirmed bytes	PREPARING, CANCELLED, EXPIRED
PREPARING	Client scan/encryption or preparation in progress	UPLOADING, FAILED, CANCELLED
UPLOADING	Bytes/chunks in transit	VERIFYING, FAILED_RETRYABLE, CANCELLED
VERIFYING	Server verifies object, size, checksum, encryption metadata and scan result	COMMITTED, QUARANTINED, FAILED
QUARANTINED	Potential malware or suspicious content; no preview/release	REVIEWED_SAFE, REJECTED, PURGED
COMMITTED	Private asset record is durable and integrity check succeeded	SUPERSEDED, RETIRED, LEGAL_HOLD
FAILED_RETRYABLE	Transient network/provider failure	UPLOADING, FAILED_FINAL, CANCELLED
FAILED_FINAL	Cannot safely complete without new upload	CREATED, PURGED
RETIRED	Not selected for future release; retention policy applies	LEGAL_HOLD or PURGED if allowed
PURGED	Object and eligible metadata removed per policy	Terminal; audit tombstone remains where required

12.3 Server-side validation

- Enforce allowlisted MIME/content signatures, maximum size, duration, page count and storage quota at the server.
- Never trust filename extension, client MIME type, client checksum or client-provided size as the only validation.
- Use quarantine until malware scanning and any required content validation completes.
- Run media parsing/transcoding in isolated workers with memory/CPU/time limits and updated dependencies.
- Use random storage object keys and never use user filename as the path or authorization token.

- Prevent path traversal, archive expansion abuse, huge page dimensions, decompression bombs and parser timeout exhaustion.
- Apply per-account quotas, upload-rate limits, concurrent upload caps and global backpressure controls.
- Do not send confidential content to third-party scanning services without approved processor contracts, data-flow review and user disclosures.

12.4 Encryption and upload trade-off

The recommended strict-confidentiality approach encrypts file content before persistent object storage, using a fresh per-asset data-encryption key. If encryption occurs solely on the client and the backend never sees plaintext, server-side malware scanning, thumbnails, OCR and video transcoding become more difficult. If server-side processing is required, the system needs a controlled processing zone that decrypts only in memory or in an isolated worker under specific purpose-bound authorization, or the product must accept reduced functionality. This is an explicit architecture decision: security claims must match the implemented flow, not marketing language.

12.5 Watermarked release derivatives

Watermarking can add a visible label with the beneficiary's name or release reference and release date to a derivative PDF or video, when appropriate and lawful. It is a deterrent and tracing aid, not a security boundary: labels can be cropped or re-recorded. Preserve the original object and its integrity hash. Log derivative generation, algorithm/version, source asset version and recipient grant. Do not alter legal originals silently or imply that watermarking prevents redistribution.

CHAPTER 13

System Architecture

Logical services, trust boundaries, deployment topology and request flow

13.1 Logical architecture

Component	Responsibility	Security boundary
Expo mobile clients	Native UI, local secure session storage, file pick/camera capture, client-side scanning/encryption where applicable	Untrusted; all values are validated server-side.
Next.js web clients	Owner experience, trustee portal, beneficiary portal and Admin Console layouts	UI visibility is not authorization.
Application API / Server Actions	Session resolution, DTO validation, authorization, domain services and orchestration	Trusted execution environment; no client receives privileged secrets.
Data Access Layer (DAL)	Centralized query methods enforcing ownership, tenant/relationship scoping and minimal DTOs	All sensitive reads and writes pass through it.
PostgreSQL / Supabase	Transactional records, relational constraints, state, metadata, audit/outbox	Database network restricted; RLS applied where relevant.
Prisma client	Typed ORM access from server-only modules	No client-side Prisma; migration/production access restricted.
Private object storage	Encrypted asset bytes, source originals, approved derivatives and evidence files	Bucket private; object ID never acts as authorization.
Key-management service	Protects key-encryption keys and mediates unwrap according to release policy	Separate service/account boundary; no raw key in logs.
Background worker	Scheduled inactivity evaluation, mail jobs, upload scans, case SLA tasks and release jobs	Idempotent, bounded concurrency, state re-check before mutation.
Notification adapter	Brevo email; future push/SMS providers	Approved templates; generic sensitive notices; delivery does not prove recipient identity.
AI Gateway	Policy gate, provider routing, redaction, schemas, quotas and usage audit	No direct model permissions to execute critical domain actions.
Observability	Redacted logs, metrics, traces, alerts and protected audit sinks	Separate operational telemetry from sensitive business/audit data.

13.2 Recommended deployment shape

- Web/API deployment: one Next.js application for user and role-specific portals with server-only domain services. If background processing or media workloads outgrow the web runtime, deploy a separate TypeScript worker from the same monorepo.
- Database: managed PostgreSQL via Supabase, with separate development, staging and production projects; production backups and network/security settings enabled according to the chosen plan.
- Object storage: private bucket/provider account with separate service credentials and environment boundaries. Ciphertext stored for high-confidentiality assets.
- Key management: provider KMS/HSM or suitable managed key service with restricted service identity, rotation and audit; avoid treating a database column as the master key vault.
- Email: Brevo sender domain authentication, verified templates, transactional sending credentials separated by environment.
- AI: outbound server-side requests only through AI Gateway; provider credentials stored in secret manager; egress allowlist if feasible.
- Edge/CDN/WAF/rate limiting: use for perimeter protection, but do not assume the edge can authorize object ownership or release policy.

- Private admin routes: require MFA/passkeys, stringent sessions and additional protections; IP allowlist or VPN may be considered for staff without relying on it as the sole control.

13.3 Request path - owner reads asset metadata

- Client calls the endpoint with session credential and asset identifier.
- Auth adapter validates session and returns canonical actor identity plus session assurance/context.
- Controller parses route/query/body against a schema and rejects unknown or oversized input.
- Domain service checks owner-resource relationship, account state, asset state, policy and requested operation.
- DAL fetches only fields needed for the client DTO; sensitive fields/secret references are excluded.
- Application emits an authorized audit event where needed, returns a minimal response and a correlation ID.
- Client renders a safe response state; error response never discloses whether another owner's asset exists.

13.4 Request path - beneficiary opens a released asset

- Beneficiary authenticates with the assurance level required by the grant.
- Backend validates ReleaseGrant status, subject, exact asset, expiry, case, hold/revocation state and current policy.
- Grant service re-checks case status immediately before authorizing delivery; stale client data is irrelevant.
- If using encrypted private storage, the release service asks key-management service to authorize unwrap for that exact asset/grant; it logs the policy decision and reason.
- Backend issues the narrowest useful time-limited read authorization or streams/decrypts within the trusted service boundary.
- Access event records metadata (actor, asset identifier, grant, time, result) without storing contents or reusable secret URLs.
- Any hold/revocation blocks subsequent access and expires/revokes grants where supported; already downloaded plaintext cannot be recalled.

13.5 Design anti-patterns prohibited

- No database credentials, Brevo API keys, AI keys, storage master keys or KMS credentials in client bundles or mobile app configuration.
- No authorization rule implemented only by hiding a button or route.
- No public object bucket for wills, IDs, death evidence or family videos.
- No generic `/admin/execute` endpoint, arbitrary SQL interface or model-controlled tool that can mutate critical data.
- No direct decryption of all assets from routine admin tooling.
- No state transition based only on an email being delivered, opened or bounced.
- No destructive database migration in production without review, backup/restore plan and staging validation.

CHAPTER 14

Technology Stack and Repository Layout

Version policy, package boundaries and coding conventions

14.1 Selected stack baseline

Layer	Choice	Implementation guidance
Language	TypeScript	Strict mode; no implicit any; runtime validation remains required.
Web	Next.js App Router	Server Components by default where appropriate; use Client Components for interaction; centralized DAL.
Mobile	React Native + Expo	Use compatible Expo SDK pinned to a tested release; app-store builds via controlled CI.
Web styling	Tailwind CSS	Shared design tokens; responsive and accessible components.
Mobile styling	NativeWind or React Native StyleSheet	Choose one primary approach and document why; avoid unmaintained styling abstraction.
Web motion	GSAP	Restrained animation; honor reduced-motion; do not animate away security status changes.
Native motion	React Native Reanimated	Use for native transitions where appropriate; keep critical action confirmations static and clear.
ORM	Prisma	Schema reviewed; migrations checked into version control; do not run ad-hoc schema push against production.
Database	PostgreSQL on Supabase	Server-side DB URL; consider transaction/pooling requirements of selected runtime.
Authentication	Better Auth	Configure trusted origins, cookie/session policy, rate limits, password/passkey/OTP options; authorization is separate.
Email	Brevo	Transactional templates, domain authentication, unsubscribe handling where required, delivery event processing.
Storage	Private object storage for sensitive assets; optional Cloudinary for approved content	Keep storage behind adapter interface.
AI	Gemini and/or Claude behind AI Gateway	Model/provider adapter; task allowlist; provider-specific data-processing review.
Validation	Zod or equivalent runtime schemas	Validate all external input and model output.
Testing	Vitest/Jest, API integration tests, Playwright, mobile test tools	Use test fixtures and isolated providers; security-specific negative tests are mandatory.
Telemetry	Structured logging + metrics/tracing provider	Redaction at source; no document contents or secrets in telemetry.

14.2 Monorepo layout proposal

The following is illustrative; exact folders may be adjusted during Stage 1. Shared packages must not import server-only secrets or Node-only APIs into mobile code.

```
portfolio-immortalized/
  apps/
    web/                    # Next.js owner, trustee, beneficiary and admin portals
    mobile/                 # Expo React Native app
    worker/                 # Optional async worker process
  packages/
    domain/                 # Pure domain types and policy definitions
    validation/             # Zod schemas and request/DTO validators
    api-client/             # Typed client API wrappers; no privileged operations
    ui-web/                 # Accessible web components and tokens
    ui-mobile/              # Native component primitives
    config/                 # Shared lint, TypeScript and formatting config
```

```
server/
  auth/                   # Better Auth server configuration
  dal/                    # Authorized database queries
  services/               # Use cases and domain orchestration
  storage/                # Object-storage adapter
  crypto/                 # Narrow interface to vetted crypto + KMS adapter
  ai-gateway/             # Provider-neutral AI service
  notifications/          # Brevo and future push/SMS adapters
  audit/                  # Domain audit writer and protected sink adapter
  jobs/                   # Scheduler, queue handlers and idempotency
prisma/
  schema.prisma
  migrations/
docs/
  adr/ security/ runbooks/ api/ threat-model/
tests/
  unit/ integration/ e2e/ security/ fixtures/
```

14.3 TypeScript and code quality policies

- Enable strict TypeScript; no blanket suppression of compiler errors in security-critical packages.
- Use branded types or validated identifiers where useful to avoid mixing UserId, AssetId, CaseId and ReleaseGrantId.
- Keep business rules in testable services rather than React components or API route handlers.
- Expose DTO types that omit secrets, internal flags, provider URLs and raw key envelopes.
- All external provider responses are untrusted input and validated before persistence or rendering.
- Run lint, typecheck, unit tests, migration checks and secret scanning in CI.
- Use code owners and mandatory review for auth, crypto, release workflow, authorization, migration and AI gateway changes.
- Pin dependency versions and maintain an SBOM or dependency inventory for production releases.

14.4 Database connectivity and Prisma

Production Prisma access must use the connection mode supported by the selected hosting/runtime and Supabase configuration. Serverless workloads may need a supported pooler URL; migration jobs commonly use a direct connection appropriate to the environment. The implementation team must confirm provider-specific connection limits and transaction behaviour before deploying. Run `prisma migrate deploy` only as a controlled release step. Do not use `prisma db push` as a production migration strategy. Apply non-destructive expand/migrate/contract changes where zero-downtime rollout is required.

CHAPTER 15

Runtime Components, Jobs and Integrations

Background work, queues, adapters and system reliability

15.1 Background job catalogue

Job family	Trigger	Idempotency key / guard	Failure handling
Inactivity evaluator	Scheduled sweep and relevant activity changes	user_id + policy_version + evaluation_window	Retry, deduplicate, record evaluation timestamp and do not duplicate notices.
Notification sender	Outbox event or scheduled reminder	outbox_event_id / provider idempotency where supported	Retry with backoff; dead-letter after bounded attempts; notify ops on critical templates.
Upload verifier	Upload complete event	upload_session_id + object version	Quarantine or retry; never mark committed without checks.
Malware/media scan	Verified upload event	asset_version_id + scanner_version	Record scanner result; fail closed for release/preview if scanner service is unavailable where required.
OCR/AI processing	Explicit user action or approved workflow	processing_job_id + input_version + task_version	Bound time/cost; store minimal output; no side effects; mark provider version.
Case SLA monitor	Periodic queue scan	case_id + SLA_rule_version + deadline	Escalate once per SLA period; delivery failure is separately tracked.
Evidence review support	Evidence upload or reviewer request	evidence_id + parser/model version	Store extracted fields as unverified; human review needed.
Release planner	Transition to AUTHORIZED	case_id + authorization_version	Re-check hold and grant plan before creating grants.
Grant issuer	Approved release plan	release_grant_id	Check case/hold each attempt; never widen asset scope on retry.
Grant expiry/revocation	Scheduled expiry or hold/security incident	grant_id + state_version	Revoke server-side; cache invalidation; record failures.
Deletion/purge	Approved deletion plan after retention check	deletion_request_id + asset/account version	Partial progress is restartable; preserve audit tombstone and report failures.
Backup/restore verification	Schedule or release gate	backup_id + verification run ID	Alert on stale backup; restore to isolated environment and validate assets/keys.

15.2 Job processing rules

- Every job payload contains opaque identifiers and minimal required data, not whole documents or secrets.
- Workers fetch authoritative state at execution time; messages are hints, not the source of truth.
- Jobs are idempotent and safe under at-least-once delivery.
- Use bounded retries with exponential backoff and jitter; classify transient versus permanent failures.
- Use per-task timeouts, maximum concurrency, queue depth limits and dead-letter review.
- Order matters for case and release jobs: validate expected state version or use compare-and-swap transitions.
- Store scheduled job state in durable infrastructure; do not depend on a single web server process staying alive.
- No job may transition a death case directly to RELEASED based solely on an elapsed timer.

15.3 Brevo integration

- Use a dedicated transactional API key per environment and minimal scopes where provider supports them.
- Authenticate sender domain and align From/Return-Path/DKIM configuration; keep secrets in a secret manager.
- All emails use versioned templates and a safe text alternative. Links are HTTPS, short-lived where applicable and point only to official app/domain.
- Sensitive subjects and previews remain generic; never include PDFs, death certificates, private videos or legacy keys as attachments.
- Webhook events are signature-verified, replay-protected, schema-validated and idempotently recorded.
- Email delivery/open/click metrics are communications metadata, not proof of human identity or proof of life.
- Support retry, bounce, complaint and suppression handling; do not retry permanent hard bounces indefinitely.

15.4 Mobile push / SMS integration options

Push notifications can be introduced through Apple/Google platform services and an approved delivery provider; SMS can be added for time-critical alerts after selecting a provider and reviewing data-transfer obligations. Device push tokens are sensitive operational data and should be stored separately from public profile fields, rotated when invalid and removed on logout/account closure as appropriate. Push notification payloads should contain an opaque event reference and generic text; the authenticated app fetches actual content. SMS OTP is not inherently phishing-resistant and should not be the sole factor for final release authorization.

15.5 Provider failure policy

- If Brevo is unavailable, queue the notification and show delivery status; do not advance safety workflow merely because a notification job was attempted.
- If storage is unavailable, retain pending upload state and do not claim the asset is saved.
- If the key-management service is unavailable, fail closed for decrypt/release, but retain case data and allow safe metadata operations.
- If AI is unavailable, show a graceful fallback and allow users/admins to proceed manually.
- If database writes fail after an external action, use idempotency keys and reconciliation jobs to align provider status and internal records.
- If a provider webhook is delayed or duplicated, state is reconciled without duplicate case transitions or repeated release notices.

CHAPTER 16

Database Architecture and Data Model

Relational design, invariants, indexes and ownership relationships

16.1 Database principles

- PostgreSQL is the source of truth for identity relationships used by the domain, asset metadata, case states, release grants, consents and audit references.
- Authentication tables managed by Better Auth remain logically separated from business Profile records, even if stored in the same database.
- Every user-owned entity includes explicit owner/tenant context and foreign-key constraints where appropriate.
- Use UUID or another high-entropy primary key consistently. Public APIs do not reveal sequential identifiers for sensitive entities.
- Store timestamps as timezone-aware UTC. Render local timezone in clients.
- Prefer explicit states and transition timestamps over loosely coupled booleans such as `is_dead`, `released` and `is_locked`.
- Keep original file metadata and current version references; store actual bytes in object storage, not large PostgreSQL bytea columns by default.
- Persist audit events separately from mutable entity rows. Never rewrite historical evidence to make current status appear as if it was always true.
- Use database uniqueness constraints for normalized email ownership where product rules require it; define trustee/beneficiary duplicates precisely.
- Use transactions for multi-record state changes and an outbox pattern for external side effects.

16.2 Entity relationship overview

At the high level, User has one Profile and many Sessions/Devices, Assets, LegacyPlans, Trustee relationships, Beneficiary relationships, Consents, Notifications, AuditEvents and SecurityEvents. TrusteeRelationship and BeneficiaryRelationship connect one owner to another person/contact and carry relationship-specific status. Asset has versions; each AssetVersion references an opaque storage object, encryption envelope and integrity record. DeathCase has evidence, reviewers, decisions, holds and zero or more release grants. Each ReleaseGrant references a single beneficiary and explicit asset set. Admin action events reference a named administrator and optionally a case.

```
User 1---1 Profile
User 1---* Asset 1---* AssetVersion
AssetVersion 1---1 StorageObjectRef
AssetVersion 1---1 KeyEnvelopeRef
User 1---* TrusteeRelationship *---1 TrusteeIdentity/User
User 1---* BeneficiaryRelationship *---1 BeneficiaryIdentity/User
User 1---* LegacyPlan 1---* LegacyInstruction
User 1---* DeathCase 1---* DeathEvidence
DeathCase 1---* CaseDecision / CaseHold / CaseEvent
DeathCase 1---* ReleaseGrant *---1 Beneficiary
ReleaseGrant 1---* ReleaseGrantAsset *---1 AssetVersion
All critical operations ---> AuditEvent + OutboxEvent
```

16.3 Core table catalogue

Table / model	Purpose	Must-have constraints / indexes
UserProfile	Progressive user profile	FK user_id unique; jurisdiction and identity status; minimal required fields.
TrusteeRelationship	Owner-to-trustee link	Unique active relation policy; invite/status indices; no implicit vault access.
BeneficiaryRelationship	Owner-to-beneficiary link	Status, contact-verification and owner-scoped lookup indices.
Asset	Logical user asset	owner_id, current_version_id, class/status; index owner + status + updated_at.
AssetVersion	Immutable version metadata	Unique asset/version_number; content hash; object reference; encryption state.
StorageObjectRef	Provider-neutral locator	provider, bucket, object key, region, object version and lifecycle state; never public URL as access control.

Table / model	Purpose	Must-have constraints / indexes
KeyEnvelope	Wrapped DEK metadata/reference	Opaque KMS key ID/version, algorithm/version, envelope bytes or secure reference; no plaintext DEK.
LegacyPlan	Versioned owner release preferences	owner_id, version, status, accepted_at, policy_version.
LegacyInstruction	Instruction/message/assignment metadata	plan_id, type, recipient scope, asset scope, release semantics and retention.
DeathCase	Formal verification case	owner_id, state, reason, current policy version, opened_at; index state + opened_at.
DeathEvidence	Evidence metadata/source object	case_id, submitted_by, type, hash, provenance, review status.
CaseDecision	Recommendation/final decision	case_id, actor, decision type, reason, policy version; unique independent-approval constraint in service/DB.
CaseHold	Hold reason/status/time	case_id, placed_by, active flag/status, lifted_by and reason.
ReleaseGrant	Beneficiary-authorized access grant	case_id, beneficiary_id, status, expires_at; revoke timestamp; no wildcard assets by default.
ReleaseGrantAsset	Assets in a grant	Composite unique(grant_id, asset_version_id); scope enforcement.
AuditEvent	Security/business history	Actor and target IDs, event type, outcome, request ID, timestamp, hash/sequence metadata.
OutboxEvent	Reliable external side-effects	Event type, payload reference, status, attempts, next_attempt_at, deduplication key.
AIProcessingJob	AI request metadata	Purpose, provider/model, policy version, consent, data class, state, retention/deletion date; no raw prompt by default.
DeletionRequest	Account/asset lifecycle deletion	Requester, scope, legal hold check, approval, execution status, purge report.
AdminRoleAssignment	Named admin permission mapping	Admin user, role, scope, assignment owner, expiry/review metadata.
ConsentRecord	Versioned consent and notices	user_id, consent_type, purpose, version, timestamp, locale, withdrawal status.
NotificationDelivery	Provider notification state	outbox ID, channel, template version, provider ID, status and timestamps.

16.4 Data integrity and transaction rules

- On case transition, check expected state and version, write transition event and append audit/outbox event in one transaction when all reside in PostgreSQL.
- When external provider calls are needed, commit an outbox event, process it asynchronously and update delivery state idempotently.
- Foreign-key delete behaviour for audit/case/evidence records should be RESTRICT or soft-retention oriented, not indiscriminate CASCADE.
- Uniqueness and check constraints should encode basic invariants, e.g. release grant expiry after creation, required reason on admin deletion, or one current asset version reference.
- Use partial unique indexes for active invitation tokens or active relationships when relevant and supported.
- Large text extracted by AI/OCR should have separate retention and data-class rules. Do not store duplicate copies in multiple tables without an owner and purge policy.

CHAPTER 17

Data Dictionary and Constraints

Representative fields and validation rules for principal domain entities

17.1 User and profile

Field	Type (logical)	Rule / purpose
user.id	UUID / opaque ID	Primary identifier; generated server-side.
profile.user_id	UUID FK	Unique; links profile to auth user.
profile.display_name	String	Required or optional according to signup configuration; length limit and output escaping.
profile.legal_name	String / nullable	Collected progressively where required; restricted field, field-level access rules.
profile.date_of_birth	Date / nullable	Collect only when necessary; validate plausible range; restrict admin access.
profile.nationality	ISO-like code / nullable	Normalized code from controlled list; avoid free-text where possible.
profile.residence_country	Code / nullable	Used for jurisdiction routing; must not be treated alone as a legal conclusion.
profile.address	Structured encrypted/restricted fields	Collect only when justified; access and retention policy documented.
profile.identity_status	Enum	NOT_REQUIRED, PENDING, VERIFIED, REJECTED, NEEDS_REVIEW.
profile.profile_version	Integer	Monotonic optimistic-lock version for edits.
profile.updated_at	Timestamp	UTC; updated by server.

17.2 Trustee and beneficiary relationships

Field	Type (logical)	Rule / purpose
relationship.id	UUID	Opaque relationship identifier.
relationship.owner_id	UUID FK	Account holder whose plan contains this relationship.
relationship.person_user_id	UUID nullable	Present after invite/account binding; must not be used as sole proof of relationship validity.
relationship.name_snapshot	Restricted string	Preserves invited name for audit when identity/profile later changes.
relationship.email_snapshot	Normalized email	Invitation destination at issue; updates require explicit flow.
relationship.phone_snapshot	E.164 string / nullable	Only when needed; verify before high-risk notices.
relationship.relationship_type	Enum/string code	Family, professional, other; free-text explanation optional.
relationship.status	Enum	INVITED, ACCEPTED, DECLINED, EXPIRED, SUSPENDED, REMOVED,

Field	Type (logical)	Rule / purpose
		UNREACHABLE.
relationship.role	Enum	TRUSTEE or BENEFICIARY; separate tables or discriminator with enforced constraints.
relationship.invitation_token_hash	Bytes / nullable	Never store raw bearer token; expires and single-use.
relationship.accepted_at	Timestamp / nullable	Set when successful transaction consumes invitation.
relationship.removed_at	Timestamp / nullable	Soft-removal marker; keep history per policy.

17.3 Asset, version and encryption metadata

Field	Type (logical)	Rule / purpose
asset.id	UUID	Opaque internal ID.
asset.owner_id	UUID FK	Owner; immutable except controlled migration.
asset.asset_type	Enum	WILL, IDENTITY, DOCUMENT, PHOTO, VIDEO, AUDIO, MESSAGE, EVIDENCE, OTHER.
asset.title	String	User-editable; escape when rendered; avoid secrets in push.
asset.sensitivity	Enum	HIGH_CONFIDENTIAL, CONFIDENTIAL, GENERAL_PRIVATE, PUBLIC_APPROVED.
asset.status	Enum	PENDING, AVAILABLE, QUARANTINED, RETIRED, LEGAL_HOLD, PURGE_PENDING, PURGED.
version.version_number	Integer	Unique and increasing per asset; immutable after commit.
version.sha256	Fixed-length bytes/hex	Hash of defined byte sequence; label whether source or ciphertext hash.
version.storage_object_id	UUID FK	References provider-neutral object metadata.
version.encryption_algorithm	Enum/version	Approved algorithm/version; never accept arbitrary client-chosen weak algorithm.
version.key_envelope_id	UUID FK	Reference to wrapped file key metadata, never plaintext key.
version.scan_status	Enum	PENDING, CLEAN, SUSPICIOUS, FAILED, NOT_APPLICABLE.
version.committed_at	Timestamp	Only set after storage, integrity and required checks pass.

17.4 Death case, evidence, decisions and grants

Field	Type (logical)	Rule / purpose
case.id	UUID	Unique, opaque case ID.

Field	Type (logical)	Rule / purpose
case.owner_id	UUID FK	Subject of investigation; must not be changed after creation.
case.state	Enum	Use the explicit state machine in Chapter 11; transitions validated by service.
case.initiation_type	Enum	INACTIVITY, TRUSTEE_REPORT, ADMIN_REPORT, EXTERNAL_NOTICE, OTHER_APPROVED.
case.policy_version	String/version	Snapshot policy used for the decision.
case.hold_reason	Separate CaseHold entity	Store multiple hold intervals; do not use one mutable string only.
evidence.source_asset_version_id	UUID FK	References immutable evidence object/version.
evidence.submitter_id	UUID FK	Person who submitted evidence; include attestation metadata.
decision.decision_type	Enum	RECOMMEND_APPROVAL, RECOMMEND_REJECTION, FINAL_APPROVAL, FINAL_REJECTION, HOLD, REOPEN.
decision.actor_admin_id	UUID FK	Named person; not a shared admin ID.
decision.reason_code	Enum	Required for material decisions; extensible policy catalogue.
decision.rationale	Restricted text	Concise explanation; warn against copying unrelated personal data.
grant.beneficiary_id	UUID FK	One recipient per grant.
grant.status	Enum	PENDING, ACTIVE, SUSPENDED, REVOKED, EXPIRED, FAILED.
grant.expires_at	Timestamp	Required unless an approved policy defines another bounded model.
grant.asset scope	Join table	Explicit versions/asset IDs; no implicit access to future uploads.

17.5 Audit event fields

Field	Purpose
event_id	Stable random identifier; supports deduplication.
occurred_at	UTC timestamp generated server-side.
actor_type / actor_id	User, trustee, beneficiary, administrator, worker, provider or system; system jobs should identify job/service identity.
action	Controlled event name, e.g. case.hold.placed, asset.upload.committed.
target_type / target_id	Resource targeted; use pseudonymous identifiers in external analytics.
outcome	SUCCESS, DENIED, FAILED, PENDING.
reason_code	Why a privileged action was taken, when applicable.
request_id / trace_id	Correlation across API, job and provider call.

Field	Purpose
policy_version	Important where an action depends on configurable policy.
integrity metadata	Sequence/hash/signed sink metadata under selected audit design; no secret material.

17.6 Index and query baseline

- Index UserProfile by normalized public identifier only where there is an explicit business need and access is restricted.
- Index Asset by owner_id/status/updated_at and AssetVersion by asset_id/version_number.
- Index TrusteeRelationship and BeneficiaryRelationship by owner_id/status and person_user_id where applicable.
- Index DeathCase by state/opened_at, owner_id/opened_at and assigned reviewer/status.
- Index ReleaseGrant by beneficiary_id/status/expires_at and case_id/status.
- Index AuditEvent by occurred_at, actor_id + occurred_at, target_type + target_id + occurred_at, and request_id; define retention/partitioning strategy before scale requires it.
- Index OutboxEvent by status/next_attempt_at and deduplication key.
- Review query plans against realistic volumes; do not add indexes that expose sensitive data through unprotected reporting views.

CHAPTER 18

API Design and Endpoint Catalogue

Versioned server-side API contract and enforcement rules

18.1 API standards

- Use a versioned prefix such as `/api/v1`; document schemas in OpenAPI from the implementation source where practical.
- Use JSON for ordinary metadata; use streaming or signed upload/download mechanisms for large bytes.
- Use opaque IDs, bounded pagination (cursor preferred for timeline data), allowlisted sorting and server-side filtering.
- Authenticate via Better Auth session integration. Derive actor from session; ignore any client-supplied actor/admin ID.
- Use idempotency keys for create operations that may be retried after network failure, especially invitations, upload completion, decisions and release requests.
- Use optimistic concurrency (`version`/ETag) for edits to high-risk configurations and case state.
- Return generic not-found/forbidden responses where revealing resource existence could be harmful.
- Validate request and response schemas; redact secrets; enforce request size/time limits.
- Provide `requestId` in responses to help support trace issues without disclosing internals.

18.2 Standard error envelope

```
{
  "error": {
    "code": "CASE_TRANSITION_NOT_ALLOWED",
    "message": "This case cannot be changed in its current state.",
    "requestId": "req_opaque_id",
    "details": [{ "field": "state", "issue": "REVIEW_REQUIRED" }]
  }
}
```

Do not include SQL errors, stack traces, secrets, other users' identifiers or raw provider responses in client-facing details. Map internal exceptions to stable error codes. A rate-limited response may include a coarse retry delay without disclosing abuse-detection thresholds.

18.3 Endpoint catalogue - identity and profile

Method / endpoint	Purpose	Controls
POST /api/v1/auth/register	Create account	Rate limit; schema validation; current terms acceptance; generic conflict response.
POST /api/v1/auth/verify-email	Consume verification token/code	One-time token, expiry, rate limiting, audit event.
GET /api/v1/me	Current actor profile/status	Returns minimal DTO, onboarding and account state.
PATCH /api/v1/me/profile	Update profile fields	Allowlisted fields, validation, version check, step-up for identity-sensitive changes.
GET /api/v1/me/security/sessions	List own sessions/devices	No session token returned; identify device conservatively.
DELETE /api/v1/me/security/sessions/:id	Revoke session	Verify session belongs to actor; server-side invalidation.
POST /api/v1/me/security/step-up	Initiate high-assurance challenge	Challenge bound to action and short expiry.
POST /api/v1/me/recovery/request	Start recovery	Risk-based flow, generic response, notification and anti-enumeration.
POST /api/v1/me/account-closure	Request account closure	Identity check, open-case/legal-hold analysis and explicit impact summary.

18.4 Endpoint catalogue - vault and uploads

Method / endpoint	Purpose	Controls
GET /api/v1/assets	List owner assets	Owner-scoped; cursor pagination; metadata only; no object keys.
POST /api/v1/uploads	Create upload intent	Quota, allowed type/size, sensitivity and upload policy.
POST /api/v1/uploads/:id/parts or provider multipart route	Upload chunk/part	Scoped upload token; limit and expiry; no global storage key.
POST /api/v1/uploads/:id/complete	Finalize transfer	Idempotent; object version, expected size/checksum and server verification.
GET /api/v1/uploads/:id	Get upload state	Must belong to actor; status and retry guidance only.
POST /api/v1/assets	Commit metadata and policy	Requires successful verified upload state.
GET /api/v1/assets/:id	Get asset metadata	Owner or explicitly authorized role; no automatic access for trustee.
PATCH /api/v1/assets/:id	Update title/category/assignment metadata	Version check; audit changes; step-up for release assignment.
POST /api/v1/assets/:id/versions	Create new version	Never overwrite legal original silently; explicit version relationship.
POST /api/v1/assets/:id/preview-authorizations	Request secure preview	Check role/state/grant; short-lived and narrow; no public URLs.
POST /api/v1/assets/:id/retire	Retire from plan	Check open case, legal hold, release assignments and confirm consequences.
POST /api/v1/assets/:id/purge-request	Request purge	Eligibility check and asynchronous controlled deletion.

18.5 Endpoint catalogue - people and legacy plan

Method / endpoint	Purpose	Controls
GET /api/v1/people/trustees	List owner's trustees	Owner scope only.
POST /api/v1/people/trustees/invitations	Invite trustee	Valid contact, quorum policy, rate limit, purpose-bound token.
POST /api/v1/invitations/:token/accept	Accept invitation	Token expiry/single use and invite recipient verification.
POST /api/v1/invitations/:id/revoke	Revoke invitation	Owner relationship permission; audit and notification.
PATCH /api/v1/people/trustees/:id	Update trustee details	Owner scope and step-up for key changes.
DELETE /api/v1/people/trustees/:id	Remove/replace trustee	Soft removal, dependency check, owner notification and audit.
GET /api/v1/people/beneficiaries	List beneficiaries	Owner scope only.
POST /api/v1/people/beneficiaries	Create beneficiary	Validate minimum required fields and duplicates.
PATCH /api/v1/people/beneficiaries/:id	Edit beneficiary	Fresh auth; exact change summary; if case open then policy review/hold.

Method / endpoint	Purpose	Controls
DELETE /api/v1/people/beneficiaries/:id	Remove beneficiary	Check current release grants/case; no retroactive erasure of history.
GET /api/v1/legacy-plan/current	Get current plan version	Owner access; version/status.
PUT /api/v1/legacy-plan/draft	Save draft instructions	Validate recipient-asset assignments; optimistic locking.
POST /api/v1/legacy-plan/confirm	Confirm current plan	Step-up authentication; snapshot policy version and exact scope.

18.6 Endpoint catalogue - cases and release

Method / endpoint	Purpose	Controls
GET /api/v1/trustee/cases	List cases assigned to trustee	Relationship scope; no unrelated cases.
POST /api/v1/cases/report	Submit death report	Assigned relationship or authorized admin; creates case, never grant.
POST /api/v1/cases/:id/evidence	Submit evidence	Case scope; file quarantine; submitter attestation; immutable source record.
POST /api/v1/cases/:id/co-sign	Provide trustee confirmation	Distinct assigned trustee, step-up auth, scoped statement.
POST /api/v1/cases/:id/owner-response	Record owner challenge/reconfirmation	Authenticated owner; immediately creates hold event.
GET /api/v1/admin/cases	List cases for admin	Role/scope check; paging and audit.
POST /api/v1/admin/cases/:id/hold	Place emergency hold	Authorized role; high-priority transaction; reason required.
POST /api/v1/admin/cases/:id/review-decision	Record recommendation/rejection	Role, checklist, rationale and state version.
POST /api/v1/admin/cases/:id/final-approval	Independent final approval	Distinct actor; step-up; quorum, hold, evidence and policy re-check.
POST /api/v1/admin/cases/:id/manual-initiation	Open investigation manually	Reason and source; uses same controlled pipeline.
GET /api/v1/beneficiary/grants	List grants for beneficiary	Own identity, active grants only as appropriate.
POST /api/v1/beneficiary/grants/:id/access	Request asset access	Identity assurance, grant state, expiry, current hold re-check.
POST /api/v1/admin/grants/:id/revoke	Revoke grant	Authorized security/release role; immediate invalidation attempt and audit.

18.7 Endpoint catalogue - administrator, audit and AI

Method / endpoint	Purpose	Controls
GET /api/v1/admin/users	Search account directory	Admin-specific permission; pagination; sensitive field minimization.
GET /api/v1/admin/users/:id/activity	View activity timeline	Audit the view; restrict security-sensitive details by role.
POST /api/v1/admin/users/:id/suspend	Suspend user	Reason; scope; effect summary; audit; notify where safe.

Method / endpoint	Purpose	Controls
POST /api/v1/admin/users/:id/restore	Restore user	Check incident/hold state; approval and reason.
POST /api/v1/admin/users/:id/delete-request	Request account purge	Retention/hold analysis; second approval when defined.
GET /api/v1/admin/audit-events	Search audit events	Read-only role; query limits; protected export path.
POST /api/v1/ai/tasks/:task	Run allowed AI operation	Server task allowlist; consent/data class/provider policy/quota checked.
GET /api/v1/ai/jobs/:id	Get AI job status	Owner or allowed case reviewer only; redact sensitive outputs.
POST /api/v1/ai/feedback	Record suggestion rating/correction	Do not feed sensitive correction to training without consent/approval.
POST /api/v1/admin/ai/disable-feature	Emergency disable AI task/provider	Super-admin/security permission, reason, audit and alert.

18.8 Example: create upload intent

Illustrative contract; exact schema and provider details are implementation decisions. Clients do not submit a final storage key or trusted owner ID.

```
POST /api/v1/uploads
Idempotency-Key: 7a2c... (random client-generated request key)
{
  "fileName": "signed-will.pdf",
  "declaredMimeType": "application/pdf",
  "declaredSizeBytes": 2840192,
  "assetType": "WILL",
  "sensitivity": "HIGH_CONFIDENTIAL"
}

201 Created
{
  "uploadId": "upl_opaque",
  "status": "CREATED",
  "maxPartBytes": 8388608,
  "expiresAt": "2026-10-16T10:00:00Z",
  "preparation": { "encryptionRequired": true, "algorithmProfile": "asset-v1" },
  "requestId": "req_opaque"
}
```

18.9 Example: release case approval response

The response is an authorization record, not direct file bytes. Grant issuance follows as an independently checked job.

```
POST /api/v1/admin/cases/case_123/final-approval
{
  "expectedStateVersion": 7,
  "decision": "APPROVE_RELEASE",
  "reasonCode": "EVIDENCE_VERIFIED",
  "rationale": "Required review checklist completed; evidence and identity checks verified."
}

202 Accepted
{
  "caseId": "case_123",
  "state": "AUTHORIZED",
```

```
  "authorizationId": "authz_opaque",
  "releasePlanStatus": "PENDING_ISSUANCE",
  "requestId": "req_opaque"
}
```

18.10 Authorization test rule

For every endpoint above, test at least: unauthenticated request; wrong role; same role but wrong resource; revoked session; suspended account; stale state version; expired invitation/grant; replayed request; duplicate webhook/job; case on hold; conflicting owner response; provider unavailable; malformed and oversized input. API existence does not imply client availability, and no route may expose sensitive asset bytes without a separate explicit authorization path.

CHAPTER 19

Eventing, Notifications and Idempotency

Durable side effects and retry-safe state change

19.1 Transactional outbox pattern

When a database transaction changes material state and requires external effects (email, push, audit sink, AI request or release job), commit an OutboxEvent in the same transaction. A worker claims pending events, processes them with an idempotency key, records provider outcome and retries transient failures. This prevents the common error in which the database changes but the notification is lost, or the notification is sent while the transaction rolls back.

19.2 Domain event catalogue

Event	Produced when	Consumers / effects
user.registered	Account committed	Verification notice, audit, onboarding.
user.email_verified	Verification accepted	Enable eligible actions, audit.
asset.upload_committed	Storage/integrity verification complete	Update UI state, asset indexing, optional approved processing.
trustee.invitation_created	Invite persisted	Brevo email, invitation-delivery tracking.
relationship.changed	Trustee/beneficiary relation updated	Owner notice, audit, legacy plan readiness recalculation.
owner.activity_verified	Accepted meaningful activity occurred	Update activity timestamp, cancel/close eligible inactivity escalation under policy.
inactivity.threshold_reached	Policy evaluation crosses threshold	Create/advance concern workflow and notifications; never release.
death.case_opened	Claim/investigation created	Admin alert, trustee task, case SLA start.
death.case_held	Hold placed	Cancel pending jobs, revoke grants where possible, alert security/reviewer.
death.evidence_submitted	Evidence uploaded and registered	Create review task, scanning and optional approved extraction.
death.case_approved	Final authorization completes	Create scoped release plan and independent issuance jobs.
release.grant_issued	Grant passes checks	Beneficiary generic notice, audit.
release.grant_revoked	Hold, expiry, dispute or compromise	Invalidate access, queue cache/provider revocation and alert.
account.deletion_requested	Deletion workflow started	Retention check, approval queue, user notice where safe.
ai.job_completed	AI task response validated	Display suggestion or create human review task; never perform sensitive write automatically.

19.3 Notification classes

Class	Examples	Policy
Transactional	Account verification, trustee invitation, password/reset/recovery notice	Short-lived tokens; generic content; delivery errors handled.

Class	Examples	Policy
Security-critical	New device, recovery changed, suspicious login, death case opened, release request, admin change	Priority and escalation; disclose minimum facts; no marketing content.
Operational	Upload complete/failure, profile checklist, backup/admin queue alerts	Retryable and deduplicated.
Marketing/newsletter	Optional product news	Separate legal basis/consent and unsubscribe rules; never intermix with critical workflow alerts.

19.4 Notification idempotency and timing

- Use a unique key such as event ID + recipient ID + template version + purpose to prevent duplicate sends.
- Keep delivery attempt history with provider message ID, outcome, retry count and next attempt.
- Use exponential backoff for transient errors and a dead-letter state for persistent failures.
- Case-state transitions are committed independently from the notification; if delivery fails, escalate communications but do not infer user status.
- Critical message retries should not create alert storms; coalesce repeated events and define escalation intervals.
- Provide a central template registry; edits to legal/security templates require review and version records.
- Do not put legacy access keys, decrypted asset links, IDs, death certificates or detailed case facts in email/SMS body.

CHAPTER 20

Security Architecture and Threat Model

Threat-driven controls for a high-confidentiality digital legacy service

20.1 Security objectives

- Confidentiality: unauthorized users, ordinary staff, compromised links, storage-provider viewers and AI systems should not be able to obtain plaintext legacy materials.
- Integrity: uploaded originals, release instructions, evidence and case decisions should be tamper-evident; changes must be attributable.
- Availability: authorized owners and beneficiaries should be able to retrieve permitted data through provider outages and recovery events.
- Authenticity: actors, invitations, approvals and material actions are attributable to a verified identity with suitable assurance.
- Non-repudiation support: audit trails and independent approvals provide accountability, although the application cannot guarantee legal non-repudiation by software alone.
- Least privilege: users, trustees, beneficiaries, admins, AI models and background jobs receive only the access required for a task.
- Safe failure: ambiguity, stale state, missing evidence, provider outage or conflicting signals stops release rather than bypasses controls.

20.2 Threat actors and assets

Actor / failure	Assets or goals at risk	Primary controls
External attacker	Account takeover, identity data, wills/videos, release manipulation	Strong auth, rate limits, session revocation, object-level authorization, encryption, monitoring.
Malicious trustee	False death claim or coercive access	Independent trustee quorum, evidence validation, owner alerts, review, hold and audit.
Malicious beneficiary	Access beyond granted items or redistribution	Per-grant scope, short-lived access, watermark derivatives, audit and clear limitations.
Compromised user device	Session theft, local cache exposure, altered relationships	Secure local storage, app lock, step-up for critical changes, device revoke and local-data minimization.
Insider / malicious admin	Unauthorized case decision or file disclosure	Least privilege, named accounts, two-person approval, break-glass audit, restricted decryption.
Cloud/provider compromise	Bulk object or metadata exposure	Application-level encryption, provider boundary, narrow credentials, independent backups and key custody.
AI prompt injection	Model reveals secrets or follows embedded malicious document instructions	No secret in prompts, system-level boundaries, input isolation, output validation, no tools for critical actions.
Software supply-chain attacker	Malicious dependency or build artifact	Lockfiles, dependency review, SBOM, CI secret scanning, signed builds where feasible.
Operational failure	Accidental deletion, duplicate release, corrupt migration	Backups, idempotency, state machine, migration review, restore drills.
Legal/process ambiguity	Invalid or disputed release	Documented jurisdiction policy, qualified legal review, human verification, appeal/dispute hold.

20.3 Threat modelling method

Use a lightweight STRIDE-style review plus abuse-case walkthroughs for each feature. For each asset and trust boundary identify spoofing, tampering, repudiation, information disclosure, denial of service and privilege escalation threats. Threat models are living artifacts: update them when adding a storage vendor, AI provider, new release rule, biometric flow, admin role or data transfer jurisdiction. A high-risk change requires a security review before release, not after an incident.

20.4 Security baseline controls

- Enforce TLS for all network paths and use secure provider endpoints; prohibit mixed-content assets and plaintext credentials.
- Use Content Security Policy, security headers, clickjacking protection, HSTS after domain configuration is verified, and controlled CORS/trusted origins.

- Validate and encode output appropriate to HTML, SQL, URL, shell and log contexts; use ORM parameters rather than string-built SQL.
- Rate-limit authentication, verification, recovery, invitation, upload and AI routes by account, device signal, IP/risk band and global capacity as appropriate.
- Use managed secrets, scoped service identities, environment separation and scheduled credential rotation.
- Use malware scanning and isolated parsing for high-risk uploads; do not run untrusted media parsing with full application credentials.
- Perform SAST, dependency scanning, secret scanning, DAST and periodic penetration testing; triage findings based on business risk.
- Protect audit logs from mutation and restrict export. Separate application logs from durable case/audit records.
- Test direct object API access, cache bypass, signed URL leakage, cross-account IDOR and stale release grants.
- Have a documented vulnerability disclosure/contact channel and security patch process.

20.5 Security acceptance gates

Gate	Pass criteria
Architecture gate	Threat model, data-flow diagram, key management design, administrator privilege design and privacy assessment reviewed.
Pre-beta gate	No unresolved Critical/High exploitable findings; auth/authorization negative tests pass; secrets scanned; backup restore demonstrated.
Pre-release gate	External penetration test or independent review for critical workflows; evidence/release fault-injection tests pass; incident runbook tested.
Ongoing gate	Critical vulnerability response SLA, recurring role review, dependency patching, quarterly privileged access sample and annual threat-model refresh.

CHAPTER 21

Authentication, Sessions and Device Security

Better Auth policy, web cookies, mobile storage, biometrics and recovery

21.1 Authentication factors

Better Auth is the proposed authentication framework, but exact plugins and flows must be validated against the chosen versions and deployment. Support a secure primary sign-in method and strongly encourage passkeys where practical. Email OTP can be used for verification and some recovery flows, but email-only OTP is weaker than phishing-resistant factors. Administrators must use MFA/passkeys. High-risk owner actions should require recent, action-bound step-up authentication. Avoid security questions as the only recovery factor because answers are often guessable or discoverable.

21.2 Web session policy

- Use server-issued session identifiers with adequate entropy and secure rotation/revocation behaviour.
- Web session cookie should be `Secure`, `HttpOnly` and have an intentional `SameSite` value based on app flows; set a restrictive Path and appropriate domain scope.
- Use CSRF defenses for state-changing cookie-authenticated requests and validate Origin/Referer where appropriate.
- Never expose raw session token values to application JavaScript unless the auth design explicitly requires it and risk review accepts it.
- Do not store bearer session tokens in localStorage for the web app.
- Set idle and absolute lifetime based on risk. Step-up challenges should have a short lifetime and be bound to the action/resource.
- Rotate or invalidate sessions after password/passkey changes, recovery completion, suspicious events, admin role changes and account closure as policy requires.
- Use generic login/recovery responses to limit user enumeration; provide useful UX without revealing account state to unauthenticated parties.

21.3 Mobile session/device policy

- Use the Better Auth Expo integration or an equivalently reviewed supported integration, with session storage backed by Expo SecureStore/platform secure storage.
- Deep-link OAuth and invitation callbacks must validate scheme, trusted domain and one-time state/nonce; never accept arbitrary return URLs.
- Never embed backend API keys, service-role keys, KMS keys or AI credentials in the mobile bundle.
- Biometric unlock uses OS-level APIs; the app receives an authentication result, not raw biometric templates. Device capability is not equivalent to account-level authorization.
- If a local PIN is offered, implement platform-aware secure storage and throttling where feasible; do not treat the PIN as a substitute for server authentication.
- On logout, clear cached sensitive metadata and invalidate server sessions. Cryptographic erasure of a local cache should be included if offline files are ever supported.
- Handle device backup/restore carefully so session secrets or sensitive file caches are not unintentionally backed up to consumer cloud backup.
- Register devices only for useful security purposes and provide a clear device-revocation UI.

21.4 Step-up authentication policy

Operation	Minimum proposed assurance
View routine profile/preferences	Active authenticated session.
Upload/review a new asset	Active session; additional check if account risk is elevated.
Change email, phone or recovery factor	Fresh authentication + verified new channel; notify old verified channel where safe.
Add/remove trustee or beneficiary	Fresh authentication; transaction summary; risk-based delay or confirmation.
Change asset-to-beneficiary release assignment	Fresh authentication; display exact before/after summary.

Operation	Minimum proposed assurance
Export/download a high-confidentiality original while owner alive	Fresh authentication; per-asset authorization and security event.
Admin role assignment or policy change	Administrator MFA/passkey + peer review/second-person confirmation.
Case decision or final release approval	Admin step-up; independent actor separation; case-specific decision confirmation.
Key release/unwrapping	Only release service under KMS/HSM policy; no direct browser request or ordinary staff credential.

21.5 Recovery safeguards

- Recovery process must consider lost device, inaccessible email, compromised email, changed phone, deceased owner and suspected trustee collusion as separate cases.
- Where possible, notify previously verified channels about sensitive recovery or credential replacement; if the channel itself is compromised, follow an alternate escalation procedure.
- Revoke old sessions and refresh tokens after recovery. Preserve audit events; do not delete the old security timeline.
- Use cooldown or extra review for recovery followed quickly by trustee/beneficiary changes or a death report.
- Recovery cannot provide a silent way to alter beneficiary rights during an open death case. Such changes should pause and route to policy review.
- Recovery codes, if offered, are one-time, high-entropy and stored hashed; show only once and provide a clear lost-code recovery policy.

CHAPTER 22

Authorization, RLS and Administrator Controls

Server authorization, database policies and defense in depth

22.1 Authorization model

Use role-based access control (RBAC) for broad role capabilities combined with relationship/resource authorization (often called ReBAC or object-level policy checks). A role such as TRUSTEE is not sufficient to access every trustee-related object: the requester must be an active trustee for that owner and the exact case. A Release Approver role is not sufficient if the person initiated the case or has a conflict under separation-of-duties rules. Every endpoint and data query must use the DAL policy methods.

22.2 Authorization decision inputs

- Canonical actor ID and active account status from verified server session.
- Current role and scope from server-controlled assignment records, not client claims.
- Resource owner ID and relationship link status.
- Resource status, sensitivity, legal hold, death-case state and release-grant state.
- Session age/assurance, step-up challenge outcome and risk flags.
- Policy version and separation-of-duties constraints.
- Requested action and exact object set. Bulk operations must authorize each object, not just the first one.
- Explicit consent or documented purpose when the task involves optional AI processing or sensitive content viewing.

22.3 Supabase RLS policy strategy

Enable Row Level Security on tables exposed through Supabase client surfaces and use restrictive policies where feasible. Because the recommended architecture accesses the database through a trusted server-side Prisma layer, do not assume RLS automatically protects every query made with privileged database credentials. Service-role keys bypass RLS and must never be shipped to clients. Use restricted server credentials, explicit DAL authorization, database grants and RLS as complementary controls, not substitutes for one another. If a browser/mobile client directly accesses any Supabase table or Storage path, that table/path needs a dedicated policy tested against anonymous, authenticated, owner, trustee, beneficiary and revoked-user cases.

22.4 Row-level policy examples in words

- Owner can read/update their own profile, subject to protected fields and validation.
- Owner can read asset metadata only when `asset.owner_id` matches the authenticated user; the user cannot set owner ID through a client update.
- Trustee can read only cases linked to an active trustee relationship and can submit evidence only while permitted by case status.
- Beneficiary can list only grants addressed to their authenticated beneficiary identity and cannot change grant scope/status.
- Admin access is mediated by distinct server-side permissions and an audit event; ordinary authenticated-user RLS must not imply admin access.
- No direct client SELECT on key envelopes, wrapped key payloads, unrestricted audit records, private object path secrets or internal risk scores.
- RLS tests must include deliberately malicious updates to owner_id, case_id, role and recipient_id fields.

22.5 Administrator operations authorization

- Use an explicit permission catalogue, e.g. `users.read`, `users.suspend`, `relationships.correct`, `cases.review`, `cases.hold`, `releases.approve`, `audit.export`, `ai.disable`.
- Assign permissions through named admin roles and optional scopes; default deny if a permission is not found.
- Record the permission decision, actor, target and reason for highly sensitive actions. Avoid making logs so verbose that they contain unnecessary private details.
- Require re-authentication for role grants/removal, policy changes, account purge and final release decisions.
- Protect the bootstrap Super Admin process with a one-time setup command/controlled setup workflow; never hardcode credentials.
- Provide periodic access review and immediate offboarding/revocation when staff changes roles or leave the company.

CHAPTER 23

Cryptography, Key Management and Recovery

Encryption design and the difficult post-death recovery requirement

23.1 The confidentiality model decision

Portfolio Immortalized needs to make a clear choice between strict end-to-end encryption in which only the owner controls decryption keys, and service-assisted recoverability that can release keys to beneficiaries after an approved death case. These goals conflict unless a carefully designed multi-party/recovery scheme is used. The recommended baseline is a controlled envelope-encryption design with cryptographic separation, restricted KMS/HSM access and a release service gated by independent approvals. Do not market this as zero-knowledge encryption if the service or its key-management authority can ultimately unwrap the file key.

Approval required Before production, document which trust entities can ever obtain plaintext, including the service operator, KMS provider, worker processes, backup operators and AI processors. If the company must be technically unable to decrypt any asset, the release and recovery feature must be redesigned around user-held, trustee-held or threshold key shares and the operational limitations clearly explained.

23.2 Proposed envelope-encryption pattern

- Generate a cryptographically secure random data-encryption key (DEK) for each asset version or an equivalently narrow cryptographic unit.
- Encrypt file bytes using a vetted authenticated-encryption scheme such as AES-256-GCM or ChaCha20-Poly1305 from a maintained cryptographic library, with unique nonces and required algorithm-specific limits.
- Bind essential metadata to authenticated data where appropriate (e.g. asset/version identity and format/version tags) while avoiding unstable fields that make recovery impossible.
- Wrap the DEK using a key-encryption key (KEK) managed in KMS/HSM or equivalent key service. Store only the wrapped DEK and key-version reference with metadata.
- Separate storage of ciphertext, wrapped key envelope and authorization decisions as far as practical; use distinct roles and credentials.
- Use an approved random source; never derive keys from user PIN, email, phone, filenames or a model-generated value.
- Authenticate ciphertext before exposing plaintext. Integrity failure should produce a fail-closed error and security event.
- Version algorithms and envelope formats. Key rotation procedure must explain whether to rewrap DEKs or re-encrypt asset data, and how both paths are tested.

23.3 Key separation and access policy

Key/material	Location	Permitted access
Per-asset DEK	In memory only during authorized crypto operation; stored wrapped	Crypto service/worker under narrow authorization; never returned to ordinary client or logged.
KMS/HSM KEK	Managed KMS/HSM	Dedicated service identity; restrict unwrap by operation, environment, audit policy and key version.
Authentication secrets	Secret manager/auth provider storage	Auth service only; independent from file encryption keys.
AI provider keys	Secret manager	AI Gateway only; not available to general app route or model prompt.
Storage credentials	Secret manager / workload identity	Upload/download adapter only; bucket/path scope minimum necessary.
Backup encryption keys	Separate backup/security domain	Recovery personnel under dual control; not identical to active application key where avoidable.
Recovery/escrow shares if used	Separate protected custody	Defined quorum and documented recovery procedure; no single routine admin can reconstruct silently.
Signing/audit integrity keys	KMS/HSM or separate key boundary	Audit/signing service only; rotation and verification procedure maintained.

23.4 Key release flow

- Client asks for a grant-scoped asset read; it does not ask directly for a DEK.

- Grant service verifies beneficiary identity, asset membership, active case authorization, expiry and current hold/revocation state.
- Policy engine produces a signed/traceable authorization decision scoped to one asset version and release grant.
- Key service verifies the calling service identity and policy token, unwraps only the required DEK and returns it only into a protected processing boundary.
- If browser/app-side decryption is selected, the security design must explain how the beneficiary receives the DEK and why that does not expose it to unrelated parties. Short-lived delivery tokens alone do not solve key secrecy.
- Record key operation metadata without recording DEK value, plaintext content or sensitive unwrapped response.
- Re-check active holds during long-running release jobs and avoid caching plaintext/decrypted keys across users or grants.

23.5 Server-side versus client-side encryption trade-offs

Approach	Advantages	Trade-offs / residual risks
Client-side encrypt, server cannot decrypt	Strong confidentiality against application-server compromise	Harder to scan/preview/transcode; post-death release requires owner/trustee threshold recovery; device key loss can make data unrecoverable.
Server-side encrypt with KMS	Straightforward scanning and media processing; easier controlled release	Privileged backend/KMS path may decrypt; high value target; must use separate services and strict audit.
Hybrid: client encrypts, trusted processing enclave/worker receives limited key authorization	Can constrain where plaintext exists while supporting selected processing	Higher complexity; enclave/isolation claims need proof; key delivery and debugging must be reviewed.
Threshold escrow / multi-party recovery	No single party needs to hold the entire recovery capability if properly designed	More complex operations, trustee availability and key-share recovery; cryptographic expert design and routine drills required.

23.6 Cryptographic implementation rules

- Use established library APIs; do not implement AES modes, random number generation, KDFs, signatures or token cryptography manually.
- Define nonce generation and collision controls for the chosen AEAD library; document maximum message/file size and chunking construction.
- For large files, use a reviewed chunked authenticated-encryption scheme with per-chunk nonce discipline, ordering/integrity and truncation detection.
- Authenticate metadata and object/version binding so ciphertext cannot be swapped between asset records without detection.
- Use constant-time comparison APIs for secret comparisons where relevant; avoid leaking key-validation detail in error messages.
- Never send keys through Brevo, chat, support tickets, ordinary application logs or an AI prompt.
- Plan key rotation, compromise response, backup decryption and provider migration before upload launch.
- Regularly test a randomly selected encrypted backup through full restoration, including key access and metadata consistency.

CHAPTER 24

File Storage, Delivery and Cloudinary Decision

Private object-store design and the role of a media CDN

24.1 Storage recommendation

Use a private object-storage provider through a provider-neutral StorageAdapter. Supabase Storage can be considered for private objects protected by access policies, or an S3-compatible provider may be selected based on KMS capabilities, region, pricing, lifecycle and multipart upload needs. Sensitive legal files and family media should be encrypted before durable storage according to the approved encryption architecture. Storage path secrecy and expiring URLs are secondary controls, not substitutes for encryption and authorization.

24.2 Cloudinary role

Cloudinary may be useful for approved public website assets, logos, marketing imagery and non-sensitive media that benefits from optimized delivery. Cloudinary documentation states that upload delivery is publicly accessible by default; it also offers private/authenticated delivery and access-controlled assets with plan-dependent features. Even restricted delivery URLs can be shared while valid. Therefore, do not treat Cloudinary as automatically "super safe" for wills simply because a URL is signed. If Cloudinary is used for confidential media, explicitly configure access controls, transformation restrictions, no-index headers where appropriate, short-lived delivery and an approved data-processing model; additionally evaluate whether application-level encryption is compatible with required transformations.

24.3 Storage key/path conventions

- Use random, opaque object keys that do not disclose legal names, email addresses or predictable user IDs.
- Separate environments and data classes into distinct buckets or namespaces with separate access policies.
- Keep originals, encrypted objects, previews, watermarked derivatives and case evidence distinguishable by metadata and retention policy.
- Never expose a provider admin API key or service-role key to browser/mobile code.
- Create private buckets by default. Any public bucket requires a specific approval and content classification.
- Ensure asset listing is authorized separately from object read; inability to fetch an object should not be the only protection against object enumeration.
- Do not cache decrypted content at a public CDN. Configure cache-control appropriately for confidential responses.
- Log provider object version and checksum needed for integrity/reconciliation, but not temporary authorization URLs.

24.4 Delivery patterns

Pattern	Use	Safeguards
Backend proxy/stream	Highest-control route for particularly sensitive assets or key-release flows	Capacity and bandwidth costs; check grant before stream; protect memory and logs.
Short-lived signed object URL	Only where confidentiality model and provider support acceptable scoped access	Short TTL, exact object/version, never logged, no broad list permission, revoke/hold limitations documented.
Client-side ciphertext download	If approved model allows beneficiary-side decryption	Key-delivery protocol and recipient isolation need independent design review.
Authenticated CDN delivery	Optional Cloudinary/other CDN for compatible media	No public derived assets; test transformation URLs, link sharing, cache controls and expiration.
Watermarked derivative	Recipient copy of an approved asset	Never replace original; audit derivation; watermark is a deterrent, not access control.

24.5 Data migration and deletion

- Asset move between providers must preserve source hashes and map object version / key envelope consistently.
- Migration jobs must copy ciphertext or use the approved decrypt/re-encrypt procedure within a protected boundary.
- Do not consider deletion complete until primary object, derivatives, temporary processing copies and provider copies are accounted for according to retention contract.

- Backups may retain data for a declared period; cryptographic erasure may be used where technically valid, but legal counsel and security must approve what is claimed.
- Purge report records which objects/metadata were deleted, retained due to hold, or pending provider confirmation.

CHAPTER 25

Privacy, Retention and Legal Readiness

Privacy by design, Kenya-first review and jurisdiction-aware product language

25.1 Privacy data inventory

Data category	Examples	Privacy/security handling
Account data	Email, display name, auth identifiers	Minimize; access control; verified channels.
Identity data	Legal name, date of birth, ID/passport data, address	High sensitivity; purpose limitation; encryption/restricted display; explicit retention schedule.
Relationship data	Trustee and beneficiary names/contact/relationship	Collected with appropriate notice; invitation scope; update/revocation audit.
Legacy content	Wills, certificates, personal letters, family media	Highest confidentiality; encrypted storage, strict access and release gates.
Death evidence	Death certificate, obituary, source metadata, reviewer notes	Restricted case access, source integrity, retention/appeal process.
Security metadata	Sessions, device IDs, IP/risk indicators, failed attempts	Collect proportionately, restrict access and define retention.
AI processing data	Task type, source excerpt, model output, provider metadata	Data minimization, purpose control, provider terms, retention and deletion.
Audit data	Actions, approvals, access outcomes	Tamper-resistant; avoid unnecessary content/PII; access and retention controlled.
Communication metadata	Email delivery/bounce, notification endpoints	Restrict and purge according to purpose; delivery signals not identity evidence.

25.2 Privacy obligations to validate

- Confirm the applicable legal role of Pie Corp Limited, including controller/processor positions in each service scenario.
- Complete a Data Protection Impact Assessment (DPIA) before production processing likely to present high risk, especially legal files, identity documents, death evidence and external AI processing.
- Identify lawful bases and notices for each collection and processing purpose; do not rely on blanket terms acceptance for optional AI uses.
- Document data-subject access, correction, objection, portability and erasure request handling with justified limits for audit/legal claims.
- Assess where Supabase, object storage, Brevo, AI providers and observability vendors process data and whether cross-border transfer safeguards are needed.
- Execute processor agreements, retention settings, incident-notification terms and access controls with each provider.
- Review retention and legal hold requirements for will versions, evidence, case decisions, audit logs and deleted accounts.
- Appoint privacy/security responsibility, maintain a data inventory and conduct periodic control reviews.

25.3 Legal product boundaries

The system stores uploaded wills and legacy material. It must not present a file upload as proof that a will is validly executed, properly witnessed, current under applicable law, or controlling over later documents. The owner should be encouraged to consult qualified counsel concerning signature, witness, execution, revocation and jurisdiction rules. “Beneficiary” inside the product is an access-recipient role and does not necessarily establish legal heirship or ownership of an estate. Trustee role in the platform may not be identical to trustee status under trust or succession law; terms should be reviewed by counsel.

25.4 Kenya-specific legal review checklist

- Review the Data Protection Act, 2019, applicable regulations and current ODPC guidance for notice, lawful basis, security, data-subject rights, breach handling, DPIA and cross-border transfers.
- Review the Law of Succession Act and relevant cases/regulations concerning written wills, execution, signatures, witnesses and the status of digitally stored copies.

- Confirm evidence-acceptance policy for death notifications issued inside and outside Kenya and escalation for foreign jurisdiction documents.
- Review contractual role and limitation language for Pie Corp Limited, trustee duties, beneficiary access, release service and disputes.
- Review consumer-protection, electronic transaction, evidence, cybercrime and records-retention rules as relevant to the operating model.
- Approve exact user-facing terminology for stored will, verified death report, legal beneficiary, trustee, administrator and release.

25.5 Proposed retention matrix to be finalized

Record	Proposed baseline	Open considerations
Unverified invite/token	Expire quickly; purge token material after expiry	Keep limited delivery record long enough for support/security.
Routine session/security telemetry	Risk-based schedule, e.g. 90-365 days depending on event class	Legal, incident and regulatory needs; minimize IP/device retention.
Audit events	Longer protected retention according to legal/business policy	Must balance accountability with minimization and data-subject rights.
Original asset/version	While account is active or longer where user instructions/legal hold require	Deleted assets may need temporary hold or backup expiry.
Death case and evidence	Retain under documented case/appeal/legal schedule	Counsel determines evidence and dispute retention.
AI input/raw OCR	Do not retain by default beyond task completion unless justified	Task history may retain metadata and minimal redacted result.
Deleted account metadata	Purge after documented grace/hold checks	Must not cascade-delete necessary audit or legal hold records.
Backups	Provider and backup schedule with defined expiry	A purge may take effect in backups only after expiration unless crypto-erasure is reliable.

CHAPTER 26

AI Integration Architecture and Use Cases

AI Gateway, data boundaries, provider abstraction and approved task design

26.1 AI purpose statement

AI exists to improve navigation, organization, document usability and administrative efficiency. It is not an authority for legal validity, death determination, identity verification, trustworthiness, beneficiary entitlement or release authorization. All critical actions remain implemented as deterministic, server-validated domain workflows. The initial launch should begin with low-risk help content and scan assistance; sensitive evidence summarization should be introduced only after a privacy assessment and measurable evaluation.

26.2 AI Gateway responsibilities

- Authenticate the invoking user/admin and check task-specific authorization.
- Check data-class policy, purpose, consent or other documented basis and provider approval before selecting a provider.
- Minimize/redact data; remove unnecessary names/IDs and avoid whole-file transmission where excerpts are sufficient.
- Choose approved provider/model/task configuration and apply token/file-size limits, timeout and cost budget.
- Use versioned prompts and JSON/schema-constrained output where supported.
- Validate model output and label it as a suggestion; reject invalid, overlong or unsafe output.
- Never provide tools that can create a release grant, approve a case, edit beneficiary assignments, delete accounts or fetch raw keys.
- Record task ID, purpose, provider/model/version, policy/consent version, latency, token/cost estimate and outcome without ordinary retention of raw prompt/content.
- Enforce retry budgets, circuit breakers, per-user quotas and global spend alerts.
- Provide a kill switch by task and provider; disable AI without breaking core vault and case functionality.

26.3 Candidate AI tasks

Task ID	Task	Inputs allowed	Output / human control	Priority
AI-HELP-01	In-app help assistant	Approved help articles, app version, user's question with PII minimization	Cited/help-linked answer; support handoff; cannot access private file contents by default.	P1
AI-DOC-01	Document classification	User-selected document or minimum metadata; content only after consent/policy	Suggested category + confidence; user confirms before save.	P1
AI-SCAN-01	Scan quality assistance	Page images processed through approved local/provider path	Blur, rotation, missing-edge or possible missing-page hints; user decides.	P1
AI-OCR-01	OCR/extraction	User-authorized document pages or approved local OCR	Candidate text and fields; source coordinates; user/reviewer verifies.	P2
AI-CASE-01	Case summary	Authorized case records and explicitly approved evidence text	Chronology, missing checklist items, mismatches; reviewer verifies.	P2
AI-RISK-01	Security event explanation	Redacted event metadata and rules-engine alerts	Readable summary and suggested priority; deterministic rules remain primary alerting.	P2
AI-MEDIA-01	Video/audio transcription	Explicitly selected recording after consent	Transcript/captions clearly marked AI-generated; original preserved.	P2
AI-PLAN-01	Legacy organization guidance	Approved general guide and user-entered non-sensitive preferences	Checklist and plain-language suggestions; not legal advice.	P2
AI-DEATH-01	Determine whether user died	Not allowed	No model output may set death state or initiate/release assets without human workflow.	PROHIBITED
AI-RELEASE-01	Approve release or invoke KMS	No such access	Not permitted; excluded from model tool registry and API capability.	PROHIBITED
AI-TRAIN-01	Train a model on stored wills/videos	Not part of baseline	Prohibited unless separately approved by owner, counsel, privacy/security and affected user's meaningful opt-in.	PROHIBITED

26.4 Provider abstraction

Implement a provider interface such as `AiProvider.generateStructured(taskDefinition, input, options)` with Gemini and Claude adapters. Task code depends on a stable domain result contract, not provider-specific response formats. Provider/model selection is configuration controlled, versioned and validated in staging. Do not automatically fail over sensitive data to a different provider unless that provider is independently approved for the same data class and region/retention profile.

```
type AiTaskName =
  | "help.answer"
  | "document.classify"
  | "scan.quality"
  | "case.summarize";

type AiTaskContext = {
  actorId: string;
  purpose: string;
  dataClass: "PUBLIC_HELP" | "PRIVATE_METADATA" | "SENSITIVE_CONTENT";
  consentRecordId?: string;
  policyVersion: string;
  requestId: string;
};

// Illustrative contract only. Critical domain actions are intentionally absent.
interface AiGateway {
  run<T>(task: AiTaskName, input: unknown, context: AiTaskContext): Promise<ValidatedAiResult<T>>;
}
```

26.5 AI response structure

Return task-specific JSON objects validated server-side. Common metadata should include task ID, provider/model identifiers, prompt/template version, generation timestamp, input version/hash reference where allowed, structured warnings, confidence/uncertainty field if the task supports meaningful calibration, and `humanReviewRequired`. Do not treat model confidence as probability of death, legal validity or truth. A “no discrepancy found” result must not be presented as verification.

26.6 Data minimization patterns

- For app help, pass the user question and relevant approved documentation; do not attach account records.
- For document classification, send only the selected upload and only when the user asks for classification or an approved service purpose exists.
- For case summary, construct a role-authorized summary DTO and exclude unrelated asset contents, recovery secrets, key metadata and other family documents.
- For security-event explanation, prefer structured events and rule outcomes over raw logs, IP addresses or session tokens.
- For transcript, use a recording explicitly selected by the owner and disclose the external/local processing pathway.
- Do not ask AI to create or infer sensitive profiles about family members; process only information necessary to the requested task.

CHAPTER 27

AI Safety, Governance and Evaluation

Prompt injection, privacy, accuracy, quality gates and human review

27.1 Threats specific to AI

- Prompt injection contained in an uploaded PDF, image OCR text, email or legacy note attempts to override system instructions.
- Output leakage: model repeats unrelated names, IDs, private case details or hidden prompt/context.
- Hallucination: model invents a date, authority, clause, document type or discrepancy.
- Overreliance: staff treat a fluent summary as official verification.
- Cross-tenant leakage from incorrectly constructed context or cache key.
- Excessive agency: model tools allow unreviewed writes, notifications, data fetching or privileged operations.
- Provider retention or model improvement behaviour conflicts with user-facing privacy claims.
- Degraded model behavior after provider model/version change causes silent accuracy loss.
- Cost/denial-of-service abuse through repeated long documents, video transcription or token-heavy queries.

27.2 AI safety controls

- Treat all document text and OCR text as untrusted data. System instructions must state that embedded document instructions are content to analyze, not commands.
- No general-purpose browser, SQL or privileged tools exposed to models in MVP. Use explicit task-specific tools only where approved.
- Bind every AI operation to actor, tenant, purpose, task and source-object version. Do not reuse cached outputs across owners or permissions without a rigorous scope-aware key.
- Sanitize model output before rendering; encode HTML and do not execute generated code or untrusted links.
- Validate structured outputs, enum values, array lengths, citations/page references and source consistency. Reject extra fields where possible.
- For extracted facts, retain source location/page and require human verification before using them in decision workflows.
- Never convert a model score into an access-control decision, death conclusion or final case disposition.
- Use budget, concurrency and file-size limits. Long video analysis should be asynchronous, opt-in and capped.
- Pin task/model versions; run regression evaluation before switching model/provider; monitor drift and complaints.
- Maintain a rapid disable switch and manual fallback for every AI feature.

27.3 AI evaluation plan

Use case	Evaluation dataset	Metrics / gates
Help assistant	Approved FAQs and deliberately out-of-scope questions	Answer grounded in docs, correct escalation, no fabrication of policy, sensitive-data leakage tests.
Document classification	Representative user-selected files across languages/scan quality, labelled by humans	Per-class precision/recall; confusion matrix; reject/unknown rate; human correction rate.
Scan quality	Pages with controlled blur, rotation, cut edges and low light	Recall of important quality defects, false warning rate and user correction outcome.
OCR/extraction	Synthetic or explicitly approved redacted documents with known fields	Field-level precision/recall, source-page association, no fabricated values, reviewer correction rate.
Case summarization	Expert-written summaries from redacted mock cases	Omission/contradiction rate, chronology completeness, fabricated fact rate, review time; no final decisions.
Security explanation	Synthetic redacted events with known expected priorities	Alert explanation accuracy, no secret leakage and no dangerous remediation without approval.
Transcription	Consented test recordings with transcripts	Word error rate by language/accent/noise, harmful hallucination rate and clear source labelling.

27.4 Human reviewer interface

- Show extracted value alongside source image/page or quoted excerpt, not just an AI-written conclusion.

- Allow reviewer to accept, correct or reject each candidate field independently.
- Clearly mark AI-generated summaries and show when/which model produced them.
- Provide “AI unavailable” and “manual review required” paths without blocking lawful case processing.
- Require reviewer rationale for any action that materially differs from a suggested discrepancy flag, where policy requires it.
- Prevent AI summaries from hiding contrary evidence; show supporting and conflicting sources separately.

27.5 Privacy and provider governance

- Use commercial/API service terms appropriate for sensitive processing and verify retention/use terms for the actual plan, not marketing summaries.
- Confirm whether provider prompts/outputs can be used for model improvement, whether human review occurs, how abuse monitoring works and how deletion/retention applies.
- Record the provider, region/processing location, subprocessor list, retention and contractual measures in the data inventory.
- Do not claim “zero retention” without the exact service feature, agreement and eligibility confirmed.
- AI provider failover is disabled by default for sensitive-content tasks until both providers are approved for that data class.
- Obtain user notice/consent where required; optional transcription and external legal-document analysis should be user-initiated and revocable for future tasks.
- Define a deletion flow for stored transcripts, OCR output and AI processing artifacts separate from the original file retention policy.

CHAPTER 28

Error Handling and User Feedback

Consistent errors without leaking secrets or misleading users

28.1 Error taxonomy

Category	Example code	Client handling
Input validation	VALIDATION_FAILED	Show field-specific correction; do not echo unsafe raw values.
Authentication	AUTH_REQUIRED / STEP_UP_REQUIRED	Redirect to authentication/challenge; preserve only safe intended path.
Authorization	ACCESS_DENIED / NOT_FOUND	Do not expose whether another person or object exists.
Workflow state	CASE_TRANSITION_NOT_ALLOWED / ASSET_LOCKED	Explain safe next step; show current permitted action without internal rule detail.
Rate limit	RATE_LIMITED	Retry after bounded delay; avoid disclosing detection criteria.
Provider transient	STORAGE_UNAVAILABLE / EMAIL_QUEUED	Show pending/retry state; do not claim operation completed.
Integrity/security	ASSET_INTEGRITY_FAILED / UPLOAD_QUARANTINED	Stop access, preserve event and route to security/support.
Policy/consent	CONSENT_REQUIRED / DATA_CLASS_NOT_APPROVED	Explain purpose and options; do not silently change provider or scope.
Conflict	VERSION_CONFLICT / STATE_CHANGED	Refresh current state and ask user to review before resubmitting.
Internal fault	INTERNAL_ERROR	Generic message and request ID; log redacted diagnostics server-side.

28.2 Safe retry and UI behaviour

- All network failures are distinguishable from validation errors and final denials.
- An in-flight request shows progress and prevents accidental repeated submit; backend idempotency remains authoritative.
- On expired sessions, retain no sensitive form state in an unsafe location; prompt reauthentication before resubmission.
- An ambiguous upload completion should query server state rather than blindly create a new asset.
- A failed release request never displays the asset based on cached UI state; refetch current grant status.
- If an admin holds a case during review, show “case is on hold” and disable decision actions until current state is refreshed.
- Never expose raw stack traces, SQL, signed URLs, storage object names or provider diagnostics to the UI.

28.3 Incident messaging

Security messages should be accurate, actionable and minimally revealing. For example: “A security review has paused access to this legacy plan. Sign in using the official app or contact support through the verified channel.” Avoid falsely asserting that a death report is valid or revealing which trustee made a report to a suspected attacker. Notification templates should be reviewed by product, security and legal stakeholders.

CHAPTER 29

Audit Logging, Observability and Alerting

Distinguish durable audit records from operational telemetry

29.1 Event classes

Class	Examples	Storage / access pattern
Business audit	Asset uploaded, trustee changed, legacy plan confirmed	Durable, actor-attributed, owner/case scoped and retained according to policy.
Security event	Failed login burst, recovery anomaly, admin role change, key unwrap denial	Restricted security store, priority alert and incident correlation.
Operational log	Request start/end, worker retry, provider timeout	Structured, redacted, short retention where appropriate.
Metrics	API latency, queue depth, upload failure rate, notification bounce rate	Aggregated; no names/document titles as metric labels.
Trace spans	API -> DB -> storage/provider -> worker	Correlation IDs; sensitive attributes omitted.
AI governance audit	Task, provider/model, purpose, consent/policy version, outcome	No raw content by default; restricted review of exceptional samples.

29.2 Mandatory audit events

- Authentication/session: success/failure, recovery, MFA/passkey enrollment/removal, session revoke and device registration.
- Profile/security: email/phone/recovery change, user suspension/restore, account closure/deletion request.
- Relationship: trustee/beneficiary invitation, acceptance, edit, removal, reassignment and contact verification.
- Asset: upload intent, completed version, failed integrity check, preview/download authorization, assignment changes, retirement and purge.
- Case: case opening, evidence upload, trustee co-sign, review assignment, hold, owner response, recommendations, final approval/rejection, reopen and closure.
- Release: grant creation, recipient verification, first access, download, expiry, revocation, failed key unwrap and provider error.
- Administration: every privileged read of sensitive account/case details where practical, role changes, break-glass access and policy configuration changes.
- AI: task requested, input class, provider/model, policy/consent version, validation result, reviewer correction and feature disable.

29.3 Audit integrity

At minimum, restrict audit writes to a narrow service, prohibit update/delete in ordinary application paths, alert on unexpected mutation attempts, and replicate high-value events to a separately controlled destination. For stronger tamper evidence, periodically hash/sign or chain event batches and store checkpoints outside the database security boundary. Hash chaining does not prevent an administrator controlling both the database and signing key from forging history; separation of key custody and independent storage is what strengthens the design. Avoid writing secrets or full document contents to audit records.

29.4 Alert matrix

Severity	Examples	Initial response target (proposed)	Action
Critical	Unauthorized release, suspected mass data exposure, key compromise, final approval bypass	Immediate page/on-call; no silent queue only	Place affected release(s) on hold; contain, preserve evidence, invoke incident lead.
High	Admin account compromise, suspicious trustee case, repeated failed key unwrap, broad export spike	Prompt response within 15 minutes during staffed/on-call coverage	Revoke session, investigate, consider account/case hold.
Medium	Repeated failed logins on one account, notification delivery failure for case escalation, queue delay	Review within one business day or defined support SLA	Investigate, notify owner/support where appropriate, retry or remediate.

Severity	Examples	Initial response target (proposed)	Action
Low / informational	Routine invite expiry, single transient provider failure, completed restore drill	Business-hours review or dashboard	Resolve via normal operational workflow.

29.5 Operational dashboards

- HTTP error rates, latency by route, auth failure rate, DB connection saturation, worker queue lag.
- Upload starts/completions/failures, average upload time by size range, integrity failures and quarantine backlog.
- Open case counts by state, median age, overdue evidence requests, held cases and release issuance errors.
- Notification deliverability, bounce/complaint rates and critical-message retry backlog.
- Key-management availability, access-denied/unwrap-denied counts, rotation status and key-service latency.
- AI task volume, cost, latency, errors, rejected schema output, human correction rate and provider disable state.
- Backup age, last restore verification, unresolved restore errors and object/key/metadata consistency.

CHAPTER 30

Backup, Disaster Recovery and Continuity

Availability of encrypted assets and recoverability of legacy plans

30.1 Recovery objectives

Proposed initial targets are database Recovery Point Objective (RPO) of no more than 24 hours and service Recovery Time Objective (RTO) of no more than 8 hours. These are design targets to validate against actual Supabase/storage/KMS service tiers, backup plans, business expectations and budget; they are not guarantees until tested. Asset recoverability includes the database, object bytes, wrapped keys, key provider configuration, essential metadata, policy versions and release case history.

30.2 What must be backed up

- PostgreSQL data and schema/migrations, including account profile, relationships, asset metadata, cases, audit references, consents and grants.
- Private object storage originals, evidence, required derivatives and integrity metadata.
- Encryption envelope metadata and the ability to access required KMS key versions in the recovery environment.
- Immutable policy/configuration versions and notification template versions needed to explain past decisions.
- Infrastructure configuration and deployment manifests, excluding cleartext secrets.
- Audit log checkpoints and incident records stored outside the primary database boundary.
- Operational runbooks, provider recovery contacts and access to administrative break-glass process.

30.3 Backup policy

- Use provider-managed backups plus a separate approved strategy for critical asset objects and metadata.
- Encrypt backups in transit and at rest, with key custody separate from the data where possible.
- Limit who can delete backups; use immutable/WORM retention or separate account boundaries where supported.
- Ensure backups do not inadvertently remove a legal hold or restore a previously revoked release grant as active without reconciliation.
- Document retention period, region, cross-border processing and cost for each backup copy.
- Never rely on a backup that cannot be decrypted after key rotation, key revocation or provider account loss.
- Maintain a tested procedure for restoring to an isolated environment without sending notifications or granting beneficiary access.

30.4 Restore drill checklist

- Provision an isolated recovery environment and restricted break-glass credentials.
- Restore database schema and data from a selected backup; apply verified migrations only when needed.
- Restore object records/versions and check a sample or full set of object hashes according to volume.
- Validate KMS key availability and unwrap/decrypt a sample of encrypted assets with authorized recovery process.
- Cross-check asset metadata count, object count, key envelope references and case/grant state.
- Replay or reconcile outbox events carefully; disable external sends and release workers until state is validated.
- Confirm revoked grants and active holds remain effective after restoration.
- Record achieved RPO/RTO, defects, remediation owner and next drill date.

30.5 Business continuity

If the platform is unavailable for several hours, staff should have read-only access to essential runbooks and a clear process for communicating outage status without sending sensitive case details. No manual spreadsheet or email-only workaround should authorize a legacy release outside the audited state machine. If a provider outage prevents safe release, delay release and explain the operational delay through approved channels; availability pressure must not weaken authorization.

CHAPTER 31

Testing Strategy and Acceptance Matrix

Unit, integration, end-to-end, security, mobile and operational validation

31.1 Testing layers

Layer	Scope	Examples
Unit	Pure rules, validators, transitions and DTO mapping	State machine denies invalid transition; inactivity rules ignore background refresh; grant scope exact.
Database/integration	Prisma, PostgreSQL constraints, DAL and RLS policies	Cross-account reads denied; dual approver constraint; audit/outbox transaction.
API integration	Auth, validation, rate limits and state checks	Revoked session; stale state version; idempotent upload completion.
Web E2E	Owner, trustee, beneficiary, admin browser journeys	Invitation acceptance, case review, hold, release and closure.
Mobile E2E	Camera, file picker, biometric and secure storage	Scan pages, resume upload, app background/restore, session revoke.
Security	Threat-driven offensive and negative tests	IDOR, broken access control, prompt injection, signed URL leakage, CSRF/XSS.
Performance	Realistic concurrent API/DB/upload workload	DB pool saturation, worker backpressure, large media upload.
Resilience	Provider outage/retry/duplicate events	Brevo down, storage outage, KMS down, delayed webhooks, worker crash.
Accessibility	Keyboard, screen-reader and contrast checks	Accessible form errors, table headings, modal focus, dynamic text.
Operations	Backup restore, incident, alert and deployment drills	Hold during release worker, restore with active case, roll back bad release.

31.2 Mandatory security scenarios

- An owner changes the request path asset ID to another owner's asset ID: request is denied without confirming foreign asset existence.
- A trustee accesses another owner's case by guessing a case ID: request is denied and security event is recorded.
- A beneficiary modifies grant ID or asset ID in preview endpoint: only assets explicitly in their active grant remain accessible.
- A suspended user uses an old cookie: sensitive requests are denied server-side.
- A case reviewer tries final approval after making the recommendation: denied by separation-of-duties check.
- A release job begins, then Security Admin places a hold: no additional grants are issued and existing new-access authorization is revoked/disabled where possible.
- An owner responds after a case is authorized but before a grant is issued: release is paused and a review is opened.
- An upload has correct extension but malicious content type: quarantine/reject; no preview or release.
- A signed URL is copied to another browser: access limitations behave as documented; protected original is not publicly available.
- An AI document contains prompt-injection instructions: output remains limited to the task; no privileged action or unrelated disclosure occurs.
- Gemini or Claude times out: UI provides manual fallback; no workflow is advanced by timeout.
- Production service-role credentials are searched for in web/mobile bundles: none are present.

31.3 Release workflow acceptance criteria

- No release occurs from inactivity alone, regardless of configured duration.
- One trustee report cannot bypass required independent trustee confirmation or evidence checks.
- AI-generated candidate facts remain marked unverified until a reviewer checks them against source evidence.
- An owner challenge, active hold, pending dispute, compromised account or invalid evidence blocks final release.
- Final release requires distinct authorized humans and an exact list of beneficiary-asset pairs.
- Every grant is time-bounded or governed by an explicitly approved alternative, can be revoked server-side and produces an audit event.
- Repeated jobs and request retries cannot issue broader or duplicate grants.
- Admin "manual trigger" opens investigation, not instant release.

31.4 Test data hygiene

Use synthetic users, invented names and synthetic will/death-evidence samples in developer and staging environments. Production files must not be copied into staging to debug AI or OCR unless an explicitly approved, minimized and appropriately protected procedure permits it. Scrub secrets from logs and test reports. Rotate any credential accidentally committed or embedded in a fixture immediately.

CHAPTER 32

CI/CD, Environments and Release Management

Safe delivery from developer workstation to the app stores

32.1 Environments

Environment	Data policy	Access and integration
Local development	Synthetic only; no production assets or keys	Local env file ignored by Git; local/test provider adapters; local database or isolated dev Supabase.
Shared development	Synthetic or anonymized test records	Separate Supabase project, storage bucket, Brevo sandbox/test list, AI mocked or low-risk test project.
Staging/UAT	Synthetic, curated and legally approved test data	Production-like policy and deployments; restricted staff; test credentials; no real sensitive files by default.
Production	Real user data under approved policy	Named privileged access, managed secrets, backups, monitoring, change approval and incident escalation.

32.2 CI pipeline

- Install dependencies using lockfile and fail on unreviewed lockfile changes.
- Run formatting/lint, strict typecheck and unit tests.
- Run secret scan, dependency vulnerability scan and static analysis.
- Validate Prisma schema and migration safety; test migrations against a representative database version.
- Run API/authz/security regression tests including cross-tenant cases.
- Build Next.js web, worker and Expo app artifacts for intended environments.
- Generate SBOM/build metadata and preserve build logs without secrets.
- Require code-owner approvals for auth, crypto, release, RLS/policy, admin, AI gateway and data migrations.
- Deploy staging automatically or with controlled promotion; run smoke tests and workflow E2E.
- Promote immutable artifacts/config to production with approval; run migration and post-deploy smoke tests; monitor rollback triggers.

32.3 Migration strategy

- All schema changes are in reviewed migrations committed to version control.
- Back up and test restoration before high-risk production schema changes.
- Use expand-and-contract for changes requiring old and new app versions to coexist.
- Do not rename/drop columns in the same release that first stops old code using them unless compatibility has been proved.
- For enum/state-machine changes, update service validation, data migration, telemetry and regression tests together.
- Never run an automatic seed script that creates a known default production admin password.
- After deployment, validate schema version, queue health, auth cookies, storage access and critical case APIs.

32.4 Release and rollback

Every release has a changelog, migration notes, feature flag plan, test evidence, owner and rollback criteria. High-risk feature deployments should begin disabled behind a server-side feature flag. Database rollback is not always a simple inverse migration; use forward fixes and restoration planning when destructive changes cannot be safely reversed. If a case/release defect is suspected, immediately disable grant issuance and place affected cases on hold before rolling back unrelated features.

32.5 Secret management

- Use platform secret managers or protected CI environment variables; do not paste production secrets into source code, issue trackers, prompts or shared documents.
- Separate service keys by provider, environment and purpose.
- Limit build agents' production secrets to the deployment step that requires them.
- Rotate on staff departure, suspected leak, provider incident or scheduled policy; test rotation before relying on it.

- Keep public client configuration explicitly distinguished from secrets; public publishable keys must still be constrained by RLS and app design.
- Monitor secret access and disable unused credentials.

CHAPTER 33

Operational Runbooks and Incident Response

Executable response procedures for likely failure modes

33.1 Security incident lifecycle

- Detect and declare: record alert, initial severity, affected service/account/case and incident commander.
- Contain: revoke sessions/credentials, disable affected integration, place relevant cases on hold and stop release grants if warranted.
- Preserve: protect audit/events, provider logs, hashes and relevant object versions; avoid unnecessary access to file contents.
- Assess: scope accounts/data/assets affected, time window, privileges exercised, key exposure and provider dependencies.
- Eradicate and recover: patch root cause, rotate credentials/keys as appropriate, rebuild trusted artifacts and restore verified data if necessary.
- Notify: follow legal/regulatory and contractual timelines after counsel/privacy lead assessment; do not promise a timeline before fact-finding.
- Learn: produce an incident report, control improvements, owners and due dates; verify remediation with tests.

33.2 Runbook - suspected unauthorized release

- Immediately activate emergency hold for affected case(s) and disable grant issuance if scope is uncertain.
- Revoke active grants and signed tokens where possible; invalidate relevant sessions/caches; identify already accessed/downloaded content.
- Preserve case decisions, KMS events, storage delivery logs, URL issuance metadata and admin audit records.
- Identify which actor/automation approved, which policy version was applied and whether two-person separation worked.
- Rotate/revoke exposed credentials or key material if evidence supports compromise; do not rotate keys blindly without a restore plan.
- Escalate to security lead, product owner and legal/privacy contact; assess affected users and notification duties.
- Re-enable release only after root cause is fixed and independent validation passes.

33.3 Runbook - compromised admin account

- Disable the admin account and revoke all sessions immediately through a second trusted admin or break-glass procedure.
- Pause open release cases touched by the account and inspect recent role, policy, export, relationship and key-management actions.
- Compare audit events with independent sink/provider telemetry; preserve evidence.
- Reset/re-enroll authentication, review registered devices and secure the admin's email identity.
- Require independent review for any case or user changes performed within the suspected compromise window.
- Check whether production secrets, keys or artifacts were accessible; rotate affected credentials where needed.
- Document recovery and require manager/security approval before re-enabling the account.

33.4 Runbook - key-management outage or suspected key compromise

- For outage, fail closed for decryption/release while leaving safe metadata and case operations available where possible.
- Check provider status, IAM changes, quota, key version and network/configuration without exposing plaintext keys.
- For suspected compromise, disable relevant unwrap paths, place impacted releases on hold and preserve KMS audit logs.
- Use approved key rotation/rewrap or re-encryption plan only after key specialist review.
- Validate a representative backup and object sample before restoring normal release operations.
- Never create a parallel emergency plaintext-key export as an improvised workaround.

33.5 Runbook - provider/AI privacy incident

- Disable the specific AI task/provider integration using feature flags and prevent automatic fallback.
- Determine data classes and records submitted, provider/model version, retention configuration and whether human review or downstream storage occurred.
- Request provider incident information and deletion/containment according to contract while preserving evidence.
- Assess whether user notice, regulator communication or other response is required with privacy/legal advisers.
- Confirm that cached prompt/results, logs and retry queues are identified and cleaned according to policy.
- Re-enable only after approval and evidence that the misconfiguration or behaviour is addressed.

33.6 Runbook - owner alleges false death report

- Place the case on immediate hold and stop grant issuance.
- Verify the owner through a secure, risk-appropriate authentication and recovery flow; consider account compromise separately.
- Preserve the initial report, evidence, submitter identity and all actions without presuming malicious intent.
- Review whether notifications and relationship changes were triggered; reverse only after verified assessment.
- Notify the owner of the outcome and available dispute route, avoiding unnecessary disclosure of other persons' private data.
- Review controls and flag potential abuse where evidence supports it; do not automatically lock out the owner solely because they exercised a veto.

CHAPTER 34

Performance, Accessibility and Mobile Quality

Performance budgets and device-level experience

34.1 Performance measurement

- Benchmark against realistic users, assets, concurrent uploads and concurrent case reviewers rather than an empty development database.
- Report server latency separately from storage, email and AI provider latency.
- Paginate audit events and large asset libraries; avoid loading full video metadata or all records at initial dashboard render.
- Use database connection pool limits and monitor slow queries; avoid N+1 queries in admin tables.
- Upload large videos directly to private storage via a narrowly scoped upload session when that architecture is approved, rather than buffering entire files in a web server memory process.
- Use thumbnails/derivatives only where policy allows; encrypted original bytes remain protected.
- Use queue-based OCR/transcription; do not block the UI on large AI responses.
- Implement cancellation and timeout where appropriate; do not cancel audit/decision commits after an unknown response without reconciling state.

34.2 Mobile test matrix

Dimension	Coverage examples
OS versions	Minimum supported iOS/Android plus currently supported versions defined in release policy.
Devices	Small screen, large screen, low-memory mid-range device, tablet where supported.
Network	Wi-Fi, mobile data, slow/unstable network, offline transition, captive portal.
Lifecycle	Cold start, background/foreground, OS kills app mid-upload, device reboot.
Permissions	Camera/mic denied, limited photo permission, notifications disabled, biometrics enrolled/removed.
Storage	Low free space, expired upload session, corrupted cached artifact, backup/restore.
Accessibility	VoiceOver/TalkBack, dynamic text, high contrast, reduced motion and external keyboard where relevant.
Security	Rooted/jailbroken device risk treatment, screen previews, logs, deep links and app tampering posture.

34.3 Motion and accessibility

Use GSAP for web motion and React Native Reanimated or platform-native animation patterns for mobile. Honor reduced-motion preferences, avoid excessive animation in a security-sensitive app and do not use animation alone to indicate case state. Animations must not obscure emergency hold actions, upload status or error warnings. Security state changes should have explicit accessible text and be announced to assistive technologies.

34.4 Accessibility acceptance

- Every interactive control has a meaningful accessible name and state.
- All forms have persistent labels; errors are associated with fields and summarized where helpful.
- Color is not the sole signal of pending, verified, on hold, rejected or released status.
- Dialogs manage focus, support keyboard escape where safe and explain destructive actions.
- Tables include headers and a sensible reading order; responsive transformation does not lose context.
- Video/audio players include controls and captions/transcripts where available; AI-generated transcript is labelled.
- UI supports zoom/dynamic text without hiding critical controls or confirmation details.

CHAPTER 35

App Store and Play Store Readiness

Mobile release process, disclosure and operational readiness

35.1 Store readiness checklist

- Apple Developer and Google Play Console ownership/access are controlled by Pie Corp Limited or an approved company account, not an individual contractor's personal account where avoidable.
- App privacy disclosures match actual data handling, including identifiers, contact details, user content, diagnostics and optional AI processing.
- Provide a public privacy policy, terms, support contact and account/data deletion request mechanism as required by store policy.
- Justify permissions. Camera and microphone prompts occur in context. Notifications explain their purpose.
- Use official HTTPS domain/app links, universal links/app links, appropriate OAuth redirect configuration and branded sender domains.
- Test release build signing, secure configuration, crash reporting redaction, API endpoint selection and environment separation.
- Implement in-app account deletion/request experience with clear limits and legal hold/retention explanation.
- Provide app review credentials or a safe review mode without real sensitive user data or a bypass to production release approval.
- Prepare accessibility review, screenshots, app description, support details, age/content declarations and release notes.
- Have a production incident contact and rollback/feature-disable strategy before submitting.

35.2 Avoid these launch shortcuts

- Do not use real will or death-certificate samples as public app-store screenshots or test fixtures.
- Do not enable public file URLs in order to make app review easier.
- Do not hardcode a Super Admin password into a seeded database or the mobile client.
- Do not let App Review/Test mode bypass auth/authorization code that differs from production; isolate test accounts/data instead.
- Do not launch death release functionality with only happy-path testing or one-person administrator approval.
- Do not claim end-to-end encryption, absolute security, guaranteed inheritance outcomes or automated death verification unless these statements are technically and legally supportable.

CHAPTER 36

Implementation Roadmap and Definition of Done

Recommended staged implementation, milestones and acceptance gates

36.1 Delivery stages

Stage	Primary outputs	Exit criteria
0 - Discovery and ADRs	Approved requirement register, screen inventory, data-flow diagram, threat model, legal questions, cloud/provider choice	Open decisions assigned with owners; encryption/release architecture reviewed.
1 - Foundation	Monorepo, CI, environments, Next.js, Expo shell, Prisma, migrations, Better Auth, shared DTOs	Secure auth, strict typing, separate environments, first RLS/DAL test suite.
2 - Profile and relationships	Progressive profile, trustees/beneficiaries, invitation and permissions	End-to-end relationship flow; tests prove no accidental vault access.
3 - Vault & crypto	Storage adapter, encryption/key envelopes, uploads, scanning, versions, preview	Independent security review; upload/resume/hash/restore tests pass.
4 - Legacy plan	Per-asset and recipient assignment, review/confirm versioning	Changes audited, user can see exact recipient mapping, high-risk changes stepped up.
5 - Death case workflow	Inactivity evaluator, reminders, evidence, co-sign, case state machine, holds, two-person approval	No inactivity-only release; concurrency/hold/fault tests pass.
6 - Admin console	User directory, activity, admin RBAC, manual cases, deletion workflow, security center	Admin separation-of-duties and audit tests; sensitive content hidden by default.
7 - AI pilot	AI Gateway, help assistant, classification/scan assistance	Provider terms reviewed; evaluation thresholds passed; kill switch tested.
8 - Hardening and beta	Pen test, accessibility, load, disaster recovery, app-store artifacts and policies	No open Critical/High exploitable findings; restore/release workflow exercised end-to-end.
9 - Controlled production rollout	Limited cohort, monitoring, support operations, incident exercises	Operational metrics stable; support and on-call owners assigned; reviewed launch decision.

36.2 Definition of done for every feature

- User story, acceptance criteria, threat considerations and owner documented.
- UI includes empty/loading/success/error/permission-denied/offline states and accessibility labels.
- API, DTO, DAL and authorization tests included; relevant database constraints and migration reviewed.
- Audit events and notifications defined for sensitive state changes.
- Provider errors, retries, idempotency and observability are considered.
- Security/privacy review complete for new data fields, providers and permission changes.
- Documentation and operational runbook updated; rollback/disable approach known.
- CI gates pass; staging end-to-end workflow pass recorded; product owner acceptance documented.

36.3 MVP launch blockers

- Encryption/key recovery model remains undocumented or untested.
- Any path permits release from inactivity alone or gives AI release authority.
- Admin release can be approved by same actor who initiated/recommended it when independent approval is required.
- Private asset storage is publicly accessible or signed URLs are logged/leaked through analytics.
- Cross-account access-control tests fail or RLS/DAL assumptions are unverified.
- Backup restore does not restore readable ciphertext/keys/metadata and case/grant consistency.
- Account deletion can erase open-case evidence or audit records contrary to retention policy.
- No incident/hold runbook, owner response flow or release-grant revocation procedure.
- AI provider terms/data handling are not approved for the types of content being sent.
- Legal and privacy questions required for the launch jurisdiction remain unreviewed.

APPENDIX A

Open Decisions and Architecture Decision Records

Items requiring explicit sign-off; not all design choices are final until owners approve

A.1 Open decisions register

ID	Question / decision	Recommended default	Decision owner
OD-01	Can the service operator ever access plaintext through the key-management path?	Document a threat model; use KMS/HSM-protected envelope encryption with narrowly gated release; do not claim zero-knowledge.	Product owner + security architect
OD-02	What recovery mechanism makes assets available after death without a single insider being able to decrypt at will?	Evaluate multi-party/threshold escrow or KMS release service with independent authorization; independent cryptographic review required.	Security architect
OD-03	Which exact jurisdictions are supported?	Launch with Kenya-specific policy and clearly limit unsupported jurisdictions until reviewed.	Product owner + legal adviser
OD-04	What evidence counts as sufficient proof of death?	Counsel-approved evidence matrix by issuing authority/jurisdiction; AI only extracts candidate fields.	Legal + operations
OD-05	How many trustees and independent confirmations are required?	Two active, separately authenticated trustees by default for high-risk death release; documented exception policy.	Product owner + legal
OD-06	What is the inactivity reminder schedule and grace period?	Configurable versioned policy; no schedule marks user deceased.	Product owner
OD-07	One hold duration or multiple stages?	One consistent minimum policy across the app; proposed 72-hour hold after required notices, pending legal review.	Product owner + legal
OD-08	Will SMS/push be part of MVP?	Email + in-app notices first; add SMS/push after vendor/region and risk review.	Product owner + operations
OD-09	Primary private storage provider?	Choose after comparing region, KMS integration, multipart upload, lifecycle, audit, price and backup.	Engineering + security
OD-10	Use Cloudinary for any confidential assets?	No for the default high-confidentiality vault; allow only after explicit access-control/processing review.	Security + product owner
OD-11	Which Better Auth login factors/plugins?	Select supported current versions; passkeys plus secure recovery preferred; email OTP available where appropriate.	Engineering + security
OD-12	Gemini vs Claude, account/region/retention terms?	Provider-neutral gateway; start with low-risk tasks and paid/API terms review.	Product/privacy
OD-13	What AI data processing is user-opt-in?	Optional transcript and sensitive-document analysis require clear separate consent/purpose policy.	Privacy lead
OD-14	What retention duration applies to evidence/audit/closed accounts?	Create schedule after legal review; no silent purge during open case/legal hold.	Legal + privacy
OD-15	RPO/RTO target and hosting plan?	Proposed RPO 24h, RTO 8h; confirm with provider/product cost.	Operations + owner
OD-16	Who are the named release approvers and backup approvers?	Two-person approval; roles assigned to named staff with offboarding controls.	Company leadership
OD-17	Does user require offline access to originals?	No offline plaintext access in MVP; revisit only with secure encrypted-cache threat model.	Product + security
OD-18	How are disputes, court orders and family disagreements handled?	Suspend release and refer to reviewed legal/operational escalation policy.	Legal + operations
OD-19	Does watermarking apply to all release copies?	Use recipient-specific derivative where technically appropriate; preserve originals unmodified.	Product + legal
OD-20	Is there a legal will creation/editor feature?	No in MVP; securely store documents and state legal-validity limitations.	Product + legal

A.2 Architecture Decision Record template

Every material architecture choice should be recorded as an ADR and retained in `/docs/adr/`. A good ADR contains: title and ID; status (proposed/accepted/superseded); date; context/problem; options considered; decision; consequences and risks; data-class/provider impact; security/privacy review; migration/rollback plan; owner and reviewers. Change a decision by creating a new ADR, not by rewriting the historical record.

A.3 Initial ADR set

ADR	Title	Status	Decision summary
ADR-001	Mobile/web client stack	Proposed	Next.js + TypeScript web and React Native + Expo mobile.
ADR-002	Backend/data-access shape	Proposed	Shared server-side domain services/DAL with optional separate worker.
ADR-003	Database and ORM	Proposed	PostgreSQL on Supabase + Prisma, migrations committed and reviewed.
ADR-004	Auth and session policy	Proposed	Better Auth; web secure cookies; native secure storage; server authorization independent.
ADR-005	Asset encryption	Open pending review	Per-asset authenticated encryption, envelope key management, strict release gating.
ADR-006	Death verification model	Recommended	Inactivity escalation only; human evidence review and independent release authorization.
ADR-007	Admin roles	Recommended	Named individual admins, least privilege, two-person release approval.
ADR-008	Storage provider	Open pending vendor review	Private object store for sensitive content; optional Cloudinary for appropriate media.
ADR-009	AI provider abstraction	Recommended	Provider-neutral gateway with task allowlist, minimization and kill switch.
ADR-010	Audit approach	Recommended	Append-only application events plus independently protected integrity checkpoints.

APPENDIX B

Extended API Examples

Representative API payloads and stateful operations for implementation planning

B.1 Create a trustee invitation

```
POST /api/v1/people/trustees/invitations
Idempotency-Key: 2d1d...
{
  "name": "Trustee Display Name",
  "email": "trustee@example.invalid",
  "phone": "+254700000000",
  "relationshipType": "PROFESSIONAL",
  "message": "I would like to invite you to act as a trustee for my legacy plan."
}

201 Created
{
  "relationshipId": "rel_opaque",
  "status": "INVITED",
  "expiresAt": "2026-10-22T08:00:00Z",
  "deliveryStatus": "QUEUED",
  "requestId": "req_opaque"
}
```

The server determines the owner from the session. The invitation link is not returned to the owner API response. The system stores a hash/reference, delivers the token through the notification adapter, expires it and records outcomes. The invitation cannot directly grant asset access.

B.2 Trustee evidence submission

```
POST /api/v1/cases/case_opaque/evidence
{
  "uploadId": "upl_verified_evidence",
  "evidenceType": "DEATH_CERTIFICATE",
  "issuingCountry": "KE",
  "issueDate": "2026-09-30",
  "submitterDeclarationAccepted": true,
  "notes": "Submitted for independent review."
}

202 Accepted
{
  "evidenceId": "evidence_opaque",
  "reviewStatus": "PENDING_REVIEW",
  "caseState": "EVIDENCE_PENDING",
  "requestId": "req_opaque"
}
```

A submitted file remains untrusted until it passes upload verification and is reviewed. The submitter cannot set reviewStatus, evidence authenticity, caseState or an approval field in the request.

B.3 Place an emergency hold

```
POST /api/v1/admin/cases/case_opaque/hold
{
  "reasonCode": "OWNER_RESPONSE_RECEIVED",
  "rationale": "A verified owner response requires immediate investigation.",
  "scope": "ALL_PENDING_RELEASES"
}
```

```
200 OK
{
  "caseId": "case_opaque",
  "hold": { "status": "ACTIVE", "placedAt": "2026-10-15T10:22:00Z" },
  "grantsSuspensionQueued": true,
  "requestId": "req_opaque"
}
```

The hold transaction should block future grant creation before asynchronous delivery/cache revocation completes. The API should not report all provider revocations complete until the relevant adapter confirms it.

B.4 AI document classification - safe response

```
POST /api/v1/ai/tasks/document.classify
{
  "assetVersionId": "asset_version_opaque",
  "userInitiated": true,
  "purpose": "ORGANIZE_VAULT",
  "consentRecordId": "consent_opaque"
}

202 Accepted
{
  "jobId": "ai_job_opaque",
  "task": "document.classify",
  "status": "QUEUED",
  "humanConfirmationRequired": true,
  "requestId": "req_opaque"
}
```

Before the task starts, AI Gateway checks that the actor owns the asset and the selected provider is approved for its data class. The result is only a suggestion. A client cannot set `providerApproved`, turn off redaction or pass a direct prompt override.

B.5 Example asset classification result

```
{
  "jobId": "ai_job_opaque",
  "status": "COMPLETED",
  "result": {
    "suggestedType": "WILL",
    "suggestedTitle": "Signed will - draft title",
    "warnings": ["Signature page should be checked by the owner."],
    "evidenceReferences": [{ "page": 1, "kind": "document-heading" }],
    "humanReviewRequired": true
  },
  "providerMetadata": { "modelId": "approved-config-alias", "templateVersion": "classify-v1" }
}
```

Do not show provider raw prompt/response in standard user exports. Candidate title/classification should be displayed beside “Accept suggestion” and “Keep current value.” AI must never infer that the will is legally valid or current.

B.6 Account deletion request

```
POST /api/v1/admin/users/user_opaque/delete-request
{
  "scope": "ACCOUNT_AND_ELIGIBLE_PERSONAL_DATA",
  "reasonCode": "VERIFIED_USER_REQUEST",
  "rationale": "Request reviewed under account-closure policy.",
  "dryRunRequested": true
}

202 Accepted
{
```

```
  "deletionRequestId": "del_opaque",
  "status": "RETENTION_REVIEW_PENDING",
  "retentionFlags": ["OPEN_DEATH_CASE_REQUIRES_REVIEW"],
  "requestId": "req_opaque"
}
```

This is a request, not a direct delete. If a legal hold or open case exists, the service must preserve relevant records and explain the hold to authorized actors. The actual purge uses a separately checked asynchronous job.

B.7 Webhook processing contract

- Verify provider signature using the official method and constant-time comparison where applicable.
- Validate timestamp/nonce or provider replay protection where available.
- Persist raw webhook identifiers and a payload digest; avoid retaining unnecessary message body or personal data.
- Deduplicate provider event IDs and make state transitions idempotent.
- Reject unsupported event types and malformed payloads without taking business action.
- Perform reconciliation if provider event order differs from local delivery attempts.
- Never let an email-open event update owner activity or release state.

APPENDIX C

Death Case State Transition Matrix

Allowed transitions and required guards

Current state	Event/action	Next state	Required guard / side effect
NONE	Inactivity threshold reached	INACTIVITY_REVIEW	Configured policy/version; no death assertion; owner reminders queued.
NONE	Trustee/admin report accepted	REPORTED	Submitter authorized; audit/report evidence reference recorded.
INACTIVITY_REVIEW	Verified owner meaningful activity	OWNER_RECONFIRMED / CLOSED_NO_EVIDENCE	Re-check identity/risk; cancel escalation and notify case owner.
INACTIVITY_REVIEW	Escalation window passes	TRUSTEE_REVIEW	No release grant; notify active trustees and admin queue.
REPORTED	Minimum evidence metadata accepted	EVIDENCE_PENDING	File is safely stored/quarantined; case not approved.
EVIDENCE_PENDING	Required documents/checklist complete	UNDER_REVIEW	Evidence integrity and submitter authority validated.
UNDER_REVIEW	Reviewer requests more evidence	NEEDS_MORE_INFO	Reason and due date recorded; no release.
NEEDS_MORE_INFO	New evidence supplied	EVIDENCE_PENDING	New file validated; old evidence preserved.
UNDER_REVIEW	Conflicting evidence/owner challenge	DISPUTED	Immediate hold; block grant issuance.
UNDER_REVIEW	Emergency hold placed	HOLD	Hold commits immediately; alert review team and suspend pending grants.
HOLD	Hold resolved with rationale	UNDER_REVIEW	Authorized reviewer; hold resolution event; re-evaluate every policy guard.
UNDER_REVIEW	Reviewer recommends approval	READY_FOR_SECOND_REVIEW	Checklist/rationale complete; actor cannot final approve.
READY_FOR_SECOND_REVIEW	Independent approver accepts	AUTHORIZED	Step-up, actor separation, hold expiry, owner alert check, evidence and quorum revalidated.
READY_FOR_SECOND_REVIEW	Independent approver rejects	REJECTED	Reason code and appeal/reopen path recorded.
AUTHORIZED	Release planner starts	RELEASE_IN_PROGRESS	State version locked/checked; exact recipient-asset matrix created.
RELEASE_IN_PROGRESS	All scoped grants issued	RELEASED	Every grant confirmed; partial states reconciled.
RELEASE_IN_PROGRESS	Hold/revocation arrives	HOLD	Stop remaining jobs, revoke outstanding authorization where possible.
RELEASED	New dispute, fraud or legal order	HOLD / REVOKED	Prevent future access and require legal/security review; downloaded copies cannot be recalled.
REJECTED	New substantive evidence accepted	REPORTED / UNDER_REVIEW	Privileged reopening; preserve previous decision and rationale.
CLOSED_NO_EVIDENCE	New report with new evidence	REPORTED	Create new or explicitly reopened case with traceable lineage.

C.1 Transition implementation requirements

- Every command includes an expected state version so stale requests cannot overwrite newer state.
- Transitions are checked in domain services; database constraints/transactional updates enforce the final write.
- Case event and durable audit event are written atomically or via a reliable pattern.
- No endpoint accepts `nextState` as an authoritative client-supplied value.
- Any transition that can lead toward release must check for a current owner response, hold, dispute, account compromise and policy mismatch.
- For complex legal dispute, policy may require manual legal counsel clearance; a timer alone must not advance to authorization.

APPENDIX D

Security Risk Register

Initial risks, owners and planned treatments

ID	Risk	Likelihood / impact (initial)	Treatment / verification	Owner
R-01	False death determination from inactivity	M / Critical	Inactivity opens case only; human evidence review and dual release approval; test no timer-only grant.	Product + Security
R-02	Insider/admin releases assets improperly	M / Critical	Least privilege, named accounts, requester/approver separation, KMS gating, append-only audit.	Security
R-03	Key loss makes legacy assets unrecoverable	M / Critical	Approved recovery architecture, key inventory, backup restore drill and escrow procedure.	Crypto architect
R-04	Key service or application compromise exposes all files	M / Critical	Separate KMS account/IAM, per-asset DEKs, monitored unwrap, workload identity and incident plan.	Security
R-05	Cross-tenant IDOR exposes files or cases	M / Critical	Central DAL policy, scoped queries, RLS where applicable, automated negative tests and pen test.	Engineering
R-06	Public/signed object URL leaks confidential media	M / High	Private storage, no URL in logs/analytics, short scope/TTL, encryption and link-leak tests.	Storage/security
R-07	Trustee collusion/fraud	M / Critical	Independent contacts, evidence checklist, admin review, owner alerts, conflict handling and hold.	Operations + Legal
R-08	Account takeover changes recipient instructions	M / Critical	Passkeys/MFA, step-up, change alerts, cooldown/risk checks and audit.	Identity/security
R-09	Cloud/AI provider retention conflicts with promise	M / High	Provider review, data minimization, contracts, DPIA, approved data-class matrix and disable switch.	Privacy lead
R-10	Prompt injection corrupts case summary	H / High	Treat document as data; no critical tools; schema/source validation; reviewer verification.	AI lead
R-11	Hard delete erases evidence needed for dispute	M / High	Deletion lifecycle, legal hold, soft-delete, dry-run and second approval for sensitive purge.	Engineering + Legal
R-12	Duplicate job issues duplicate or broader grants	M / Critical	Idempotency keys, transactional state check, exact scope and concurrency fault tests.	Engineering
R-13	Lost phone leaves local plaintext cache	M / High	No offline plaintext in MVP; secure storage; cache minimization; logout/session revocation.	Mobile/security
R-14	Provider outage prevents notification but case advances	M / High	Delivery separate from decision; outbox/retry; overdue escalation and no delivery-based proof.	Operations
R-15	Backup cannot decrypt after key rotation	M / Critical	Key-version inventory, recovery credentials, restore exercises and independent backup boundary.	Operations/security
R-16	Excessive admin surveillance violates privacy	M / High	Purpose-based access, search/view audit, data minimization, role review, retention policy.	Privacy lead
R-17	Invalid legal assumptions for foreign will/death evidence	M / Critical	Jurisdiction-specific legal review, explicit product limitations and unsupported-jurisdiction handling.	Legal
R-18	Sensitive data exposed in email, push or telemetry	M / High	Generic templates, redaction, DLP-like tests and no content in logs/previews.	Engineering
R-19	Malicious file parser compromises service	M / High	Quarantine, malware scan, isolated worker, bounded resources and patched dependencies.	Security
R-20	Database migration causes irreversible data loss	L-M / Critical	Reviewed migrations, staging rehearsal, backups, expand/contract, restore test.	Engineering
R-21	AI model update causes accuracy regression	M / Medium-High	Pin versions, evaluation benchmark, staged rollout and feature-level rollback.	AI lead
R-22	Single Super Admin identity is lost/compromised	M / Critical	Controlled recovery/bootstrap, at least two authorized admin custodians, no shared login.	Leadership
R-23	Storage deletion incomplete across derivatives/backups	M / High	Object inventory, provider lifecycle review, purge reconciliation and reports.	Operations
R-24	Release grant cached after hold/revocation	M / Critical	Short TTL, server-side grant checks, cache purge and race-condition tests.	Engineering

ID	Risk	Likelihood / impact (initial)	Treatment / verification	Owner
R-25	Watermark falsely marketed as preventing copying	M / Medium	Clear limitation language; watermark is traceability deterrent only.	Product + Legal

Initial qualitative ratings are placeholders and must be recalculated in a workshop using the organization's agreed likelihood/impact scale. A risk is not "accepted" until an accountable owner records the decision, compensating controls and review date.

APPENDIX E

Role-Permission Matrix

High-level baseline; resource-level policy still applies to each operation

Capability	Owner	Trustee	Beneficiary	Support Admin	Security Admin	Case Reviewer	Release Approver	Super Admin	Auditor
Read own profile	Yes	Own profile	Own profile	Scoped metadata	Scoped	Case-only fields	Case-only fields	Policy-scoped	Audit only
Read owner vault metadata	Own	No by default	Assigned assets only	No by default	Metadata if incident scope	Evidence/case scope	Scope needed for decision	No default plaintext	Metadata/audit only
Upload private assets	Own	Evidence only	No	No	No	Evidence workflow only	No	No routine use	No
Manage own trustees/beneficiaries	Yes	No	No	Support correction with authorization	If incident scope	No routine change	No routine change	Policy admin only	No
Submit death report	Yes may challenge/report support	Assigned trustee	No	No	May open investigation	May open investigation	May open investigation	May under policy	No
Review death evidence	No authority over final case decision	Submit/co-sign as policy	No	No	Limited incident review	Yes	Review to approve independently	Only under assigned role/policy	Read-only if authorized
Place emergency hold	Can challenge/trigger safety hold	Can raise concern	Can dispute access	Escalate to authorized role	Yes	Yes	Yes	Yes	No
Recommend release decision	No	No	No	No	Only if explicitly assigned and independent	Yes	Not as requester	Only if assigned role	No
Final release approval	No	No	No	No	Only if assigned distinct role	No for own recommendation	Yes with independence	Only if distinct role/policy	No
Read audit events	Own relevant history	Case-related history only	Own grant/access history	Support-scope records	Security scope	Case scope	Release decision scope	Admin audit scope	Read-only approved scope
Export audit data	Own data request	No	Own access history where supported	No by default	Incident exports with reason	Case evidence report	Release decision export	Scoped/system report	Yes if assigned
Manage admin roles	No	No	No	No	No unless scoped governance role	No	No	Yes with review	No
Request account purge	Own closure request	No	No	Initiate support request	For incident needs	For case policy needs	No	Approve per policy	Audit outcome
Direct raw key access	No	No	No	No	No by default	No	No	No routine access	No
AI case summary request	Optional own content	Assigned case only if approved	No	Approved support task	Redacted incident task	Authorized case	Case review only	Task configuration only	Audit metrics only

E.1 Permission catalogue suggested IDs

- `users.read`, `users.suspend`, `users.restore`, `users.close.request`, `users.purge.approve`.
- `relationships.read`, `relationships.correct`, `relationships.remove`, `relationships.invite.resend`.
- `cases.read`, `cases.assign`, `cases.evidence.request`, `cases.review`, `cases.recommend`, `cases.hold`, `cases.reopen`.
- `releases.plan.read`, `releases.approve`, `releases.revoke`, `releases.audit`.
- `security.sessions.revoke`, `security.alerts.read`, `security.incident.manage`, `security.keyops.read` (not raw key access).
- `audit.read`, `audit.export`, `ai.jobs.read`, `ai.task.disable`, `admin.roles.assign`, `policy.configure`.

APPENDIX F

Detailed Test Catalogue

Representative acceptance, security, resilience and usability tests

Test ID	Scenario	Expected result
AUTH-01	Registration with valid email	New account created; verification required; minimal signup only.
AUTH-02	Registration with malformed email	Validation error; no account side effects.
AUTH-03	Verification token replay	Second use rejected; audit/security metric recorded.
AUTH-04	Expired verification token	No verification; safe resend flow.
AUTH-05	Brute force OTP	Rate limit and throttling enforced.
AUTH-06	Revoked session replay	Protected API denied server-side.
AUTH-07	Admin without MFA/passkey	Admin session cannot reach privileged operations.
AUTH-08	Recovery followed by beneficiary change	Risk hold/step-up policy applied; alerts sent.
PROFILE-01	Signup with only minimal fields	User reaches dashboard; progressive completion checklist shown.
PROFILE-02	Owner changes legal identity fields	Validation, restricted access and audit event created.
PROFILE-03	User asks to erase account with open case	Purge blocked/paused; retention reason and case preserved.
REL-01	Invite trustee with valid email	Expiring single-use invite queued and audit logged.
REL-02	Invite URL replay	Second acceptance rejected.
REL-03	Invite forwarded to different identity	Recipient binding policy rejects or requires re-verification.
REL-04	Trustee queries unrelated case ID	Denied; no data leakage.
REL-05	Owner removes only required trustee	System warns plan becomes incomplete and enforces policy.
REL-06	Beneficiary changes while case open	Change held/step-up/reviewed according to policy.
VAULT-01	Upload valid PDF	Verified object, hash, metadata and encryption record committed.

Test ID	Scenario	Expected result
VAULT-02	Extension spoofing	Quarantine/reject; no preview.
VAULT-03	Oversized file	Rejected before exhausting service resources.
VAULT-04	Interrupted upload	Resumes or restarts safely with no duplicate asset.
VAULT-05	Duplicate completion callback	Idempotent; one committed version.
VAULT-06	Checksum mismatch	Asset not committed; event logged.
VAULT-07	Malware scanner unavailable	Fail-closed policy applied; no release/preview where required.
VAULT-08	PDF has several pages scanned	Pages reorder/rotate; final output preview correct.
VAULT-09	Video record permission denied	Graceful denial and settings instructions.
VAULT-10	Large video upload on mobile background	Durable status and retry/resume behaviour correct.
VAULT-11	Cloudinary URL pasted to another user	No confidential public access; restriction/encryption policy verified.
VAULT-12	Replace will during open case	Version changes are controlled; reviewer sees version and plan impact.
CASE-01	Inactivity threshold passes	Concern/investigation starts; no release grant.
CASE-02	Email open event arrives	Does not reset meaningful activity.

Test ID	Scenario	Expected result
CASE-03	Phone offline for weeks	Escalation only; no death declaration.
CASE-04	One trustee reports passing	Case open; required quorum/evidence missing; no release.
CASE-05	Two trustees co-sign but evidence invalid	Human review does not auto-approve.
CASE-06	AI extracts matching name/date	Candidate values only; human verification required.
CASE-07	AI flags document mismatch	Review checklist shows source and discrepancy; no automatic rejection/approval.

Test ID	Scenario	Expected result
CASE-08	Owner verified response during hold	Case suspended; no new grants; owner not locked out unnecessarily.
CASE-09	Admin opens manual case	Investigation created; all controls remain required.
CASE-10	Admin reviewer tries own final approval	Denied by separation-of-duties rule.
CASE-11	Two approvals submitted concurrently by same person	Second cannot satisfy independent quorum.
CASE-12	Hold placed while job issues grants	Race test confirms remaining grants stopped and access revocation queued.
CASE-13	Case evidence is deleted by user	Original evidence retained/purged per legal hold/retention rules.
CASE-14	Conflicting court/legal notice reported	Case pauses and is routed to legal review.
CASE-15	Release notification fails	Case/grant state remains accurate; alert/retry; no false claim of delivery.
ADMIN-01	Support admin attempts raw file download	Denied by default; access escalation workflow required.
ADMIN-02	Admin removes beneficiary with legal hold	Operation blocked or queued for legal/release review.
ADMIN-03	Admin requests hard purge	Dry-run, checks, reasons and independent approval enforced.
ADMIN-04	Admin role changed during live session	Privileges refresh/revoke according to policy.
ADMIN-05	Break-glass use	Short-lived, case-bound, alerted and fully audited.
ADMIN-06	Admin search for high-risk identity	Search permission and audit policy enforced.
ADMIN-07	Audit event update/delete attempted	Ordinary application path rejects; alert emitted.
AI-01	AI task without consent/purpose	Request rejected before provider submission.
AI-02	AI provider times out	Manual fallback; no workflow state changes.
AI-03	Prompt injection in uploaded document	No privileged tool invoked or secret disclosed.

Test ID	Scenario	Expected result
AI-04	AI emits invalid JSON/enum	Output rejected; safe retry/fallback.
AI-05	Provider version changes	Evaluation gate required before rollout.
AI-06	AI asks to use unapproved fallback provider	Gateway denies provider switch for data class.
AI-07	AI transcript delete request	Derived data lifecycle follows retention/purge policy.
SEC-01	Cross-tenant asset IDOR	Denied, no foreign object metadata leaked.
SEC-02	CSRF state-changing request	Rejected by token/origin protections.
SEC-03	Stored XSS in filename or case note	Encoded; no script execution.
SEC-04	Query tampering / SQL injection string	Parameterized ORM/query; no injection.
SEC-05	Expired signed delivery URL	Denied after expiry; no fallback public path.
SEC-06	Service key search in mobile bundle	No privileged secret present.

Test ID	Scenario	Expected result
SEC-07	Log inspection after failure	No tokens, raw DEKs, private doc body or reusable URL.
SEC-08	Rate-limit exhaustion	Limited without cascading service outage.
OPS-01	Database unavailable	Safe error; state not falsely committed.
OPS-02	Brevo unavailable	Outbox queues/retries; case remains correct.
OPS-03	KMS unavailable	Decryption/release fails closed; metadata operations continue where safe.
OPS-04	Restore production-like backup to isolated env	Database, objects, keys and grants reconcile; outbound release disabled.
OPS-05	Duplicate outbox event	One logical notification/grant effect.
OPS-06	Migration rollback plan exercise	Recovery or forward fix works as documented.

Test ID	Scenario	Expected result
OPS-07	Critical alert simulation	On-call notification delivered and acknowledged.
MOB-01	App killed midway through scan/upload	User can recover or discard; no silent loss.
MOB-02	Biometrics removed after enrollment	Fallback works; sensitive operation requires proper reauth.
MOB-03	App switcher displays sensitive content	Privacy mitigation behaves as designed.
MOB-04	Screen reader traverses case hold controls	Names, states and focus order usable.
MOB-05	Deep link contains expired invitation token	Rejected safely; no open redirect.
MOB-06	Offline mode with expired session	No cached high-confidentiality content becomes accessible.

F.1 Test evidence required for sign-off

- Build/commit reference and environment version.
- Test case ID, tester, timestamp and result.
- Redacted logs/request IDs for failed or security-critical cases.
- Known limitation, severity, workaround and approved risk owner for any deferred failure.
- Independent security-review/penetration-test summary before release functionality moves to production.

APPENDIX G

Environment Variables and Secrets Inventory

Names are illustrative; values belong only in the appropriate secret manager

Variable / secret	Consumer	Classification	Rule
DATABASE_URL	Server/worker	Secret	Server-only. Use approved pooled URL/runtime config; no client exposure.
DIRECT_URL / migration URL	Migration runner only	High secret	Separate credential; use only for controlled migration job.
BETTER_AUTH_SECRET	Auth service	High secret	Random high entropy; rotate under documented procedure.
BETTER_AUTH_URL / trusted origins	Auth service	Config	Explicit allowlist by environment; no wildcard production origins.
BREVO_API_KEY	Notification adapter	High secret	Separate dev/staging/prod; restrict in secret manager.
BREVO_WEBHOOK_SECRET	Webhook handler	High secret	Verify signatures; never log raw signature secrets.
OBJECT_STORAGE_CREDENTIALS	Storage adapter	High secret	Workload identity where supported; minimum bucket/path scope.
KMS_KEY_REFERENCE	Crypto/key service	Sensitive configuration	Reference only; raw key material stays within KMS/HSM boundary.
KMS_SERVICE_IDENTITY	Crypto/release service	High secret/identity	Only approved service can unwrap; monitor policy changes.
GEMINI_API_KEY	AI Gateway	High secret	Server-only; provider-specific budget and data-class policy.
CLAUDE_API_KEY	AI Gateway	High secret	Server-only; no automatic sensitive-data fallback.
CLOUDINARY_CLOUD_NAME	Approved media adapter	Public-ish config	Does not confer secure access; configure delivery type and policy.
CLOUDINARY_API_KEY / SECRET	Media adapter	High secret	Server-only; restrict account permissions and rotate.
PUSH_PROVIDER_CREDENTIALS	Notification adapter	High secret	Optional future integration; split by environment.
SENTRY_DSN / telemetry key	Client/server telemetry	Config / limited secret	Apply scrubbing and avoid secrets/content in events.
ADMIN_BOOTSTRAP_TOKEN	One-time setup tool	Critical temporary secret	Not a permanent account password; expires after named admin setup.
FEATURE_FLAGS	Server-side configuration	Sensitive config	AI/release kill switches controlled server-side and audited.
PUBLIC_API_BASE_URL	Mobile/web client	Public config	No secret; environment-specific API endpoint.
PUBLIC_AUTH_CLIENT_CONFIG	Client	Public config	Only explicitly public fields; server secret excluded.

G.1 Secret handling rules

- No secret files are committed to Git; `.env.example` contains placeholders only.
- Production values are accessed only by runtime identities that need them.
- Rotate after suspected exposure; a Git delete does not remove secret history or copies.
- Validate production build bundles and mobile configuration for accidental key inclusion.
- Use different credentials for read-only reporting, upload, notification and key operations when provider supports it.
- Review secret inventory after adding/removing a provider or integration.

APPENDIX H

Requirements Traceability Matrix

Tie product requirements to implementation, test and acceptance evidence

Requirement group	Primary chapters	Primary components	Required proof
Minimal signup / profile	4, 7, 10, 21	Auth, Profile, onboarding screens	AUTH-01..08, progressive profile E2E.
Mobile-first app	6, 7, 34, 35	Expo client, native components	Mobile device matrix; app-store gate; accessibility tests.
Secure vault uploads	12, 16, 18, 23, 24	Storage adapter, upload service, crypto service	VAULT-01..12; integrity and restore tests.
Trustees / beneficiaries	3, 7, 8, 16, 18, 22	Relationship DAL, invitations, assignment editor	REL-01..06; cross-relationship authorization tests.
No “I Am Alive” button	11, 19, 31, 39	Activity recorder, inactivity scheduler, case service	CASE-01..04 and explicit UI regression test proving button absent.
Death evidence and case workflow	9, 11, 16, 18, 39	DeathCase, Evidence, Decision, Hold, release worker	CASE-04..15; state transition suite.
Admin can monitor accounts/activity	9, 18, 22, 29	Admin Console, audit service	ADMIN-01..07; admin route permission checks.
Admin can delete/manage relationship	9, 18, 25	DeletionRequest, relationship admin service	ADMIN-02/03; legal-hold and purge dry-run tests.
Admin alert / manual hold/trigger	9, 11, 19, 29, 33	Alert centre, case service, outbox	CASE-09/12; critical alert simulation.
Independent release approval	11, 18, 22, 23, 31	Decision service, KMS release service	CASE-10/11/12; separation and concurrency tests.
AI integration	26, 27, 38	AI Gateway, provider adapters, AI jobs	AI-01..07, benchmark report, kill switch test.
High-level security/encryption	20-24, 30-33	Auth/DAL/KMS/storage/audit/infra	Threat model, pen test, restore proof, incident exercise.
Brevo and notification handling	15, 19, 28, 29	Notification adapter/outbox/templates	Provider outage, webhook replay and delivery-retry tests.
Legal/privacy readiness	2, 25, 35, 36	Consent, retention, policies, public notices	Legal sign-off, DPIA, data inventory, deletion policy.
Cloud storage & Cloudinary decision	12, 23, 24	Storage adapters, access policy	No-public-object inspection, URL leakage test and provider review.
Disaster recovery	30, 31, 33	Backup, KMS, restore runbook	Isolated restore evidence and measured RPO/RTO.
App-store launch	34, 35, 36	Expo build pipeline, privacy notices	Store checklist and production config scan.

H.1 Traceability maintenance

The engineering lead should maintain a machine-readable requirement register (CSV/JSON or project tracker) with owner, priority, source label, chapter, issue/PR, test ID, evidence link, status and accepted exception. This appendix is the initial human-readable baseline; it should be refreshed as implementation decisions are made.

APPENDIX I

Glossary and Data Handling Classes

Terminology used throughout the document

Term	Definition
AEAD	Authenticated encryption with associated data; provides confidentiality and integrity when used correctly.
AI Gateway	Server-side layer governing AI provider access, task policies, data minimization, validation and auditing.
Asset	Logical item in the vault, such as a will or video; may have multiple immutable versions.
Asset version	Specific immutable uploaded revision with its own hash, object reference and encryption envelope.
Beneficiary	Person the owner designates to receive particular material under a release plan; this role does not necessarily establish legal heirship.
Break-glass	Exceptional, time-limited emergency access under stricter audit and approval controls.
DEK	Data-encryption key used to encrypt a file or defined cryptographic unit.
Death case	Formal case workflow for investigating a death report; distinct from the report itself.
Envelope encryption	Encrypt data with a DEK, then wrap the DEK with a separate key-encryption key managed by a key service.
KMS/HSM	Key-management service / hardware security module that protects cryptographic key material and operations.
Legacy plan	Versioned user configuration defining instructions, item assignments and release preferences.
Meaningful activity	Authenticated interaction accepted under policy as recent owner activity; passive telemetry alone is excluded.
Outbox pattern	Persist an event within the same DB transaction as business state, then deliver external side effects reliably.
RLS	PostgreSQL Row Level Security; a database mechanism that constrains row access when policies and grants are correctly configured.
Release grant	Scoped authorization for one beneficiary to access defined assets for an allowed period.
Trustee	Person nominated to perform defined verification/case tasks; platform role is subject to the product's terms and applicable law.
TDD	Technical Design Document; implementation-oriented description of architecture, data, APIs, workflows, security and operations.
Zero-knowledge claim	Claim that provider/operator cannot access plaintext or secrets; cannot be made if ordinary service/KMS paths can unwrap asset keys.

I.1 Proposed data classification levels

Level	Examples	Rules
C0 - Public-approved	Marketing images, public pages, approved logos	Can be public/CDN after content owner approval; still use normal integrity/access controls.
C1 - Internal operational	Non-sensitive configuration, aggregate metrics	Internal access; do not publish unless reviewed.
C2 - Private personal	Basic profile, relationship status, support metadata	Authenticated role-limited access, TLS, defined retention and minimal telemetry.
C3 - High confidential	Wills, IDs, family videos, death evidence, exact beneficiary mappings	Application-level encryption, private object storage, no AI processing by default, step-up/highly restricted access and audited disclosure.
C4 - Critical secrets	DEKs, KMS credentials, auth/session secrets, AI/storage provider keys	Never store/log/return in ordinary app data; secret manager/KMS; least privilege; rotation and incident plan.

APPENDIX J

Source References and Review Checklist

Authoritative resources used for architecture context; verify versions and terms at implementation time

J.1 Source blueprint supplied by product owner

Portfolio Immortalized - Design Blueprint.pdf, 20 pages. The blueprint establishes the original product intent: secure digital storage and controlled release, registration/profile requirements, trustees and beneficiaries, four-tab UI (Home, Portfolio, Network, Settings), an "I Am Alive" pulse-check concept, automatic and manual death trigger descriptions, evidence and administrator review, beneficiary release, watermarking, and the original proposed Flutter/React Native + React + Python FastAPI + PostgreSQL + Fernet stack. This TDD supersedes the old manual pulse-check UX and Python stack according to the revised owner requirements, while retaining the underlying custody and controlled-release aim.

J.2 External technical references

The following URLs are starting references for implementation research, not proof that a given configuration is automatically secure. Engineers should verify current documentation and plan-level details before use.

- **Next.js authentication guide:** Authentication/session management/authorization and use of a centralized data-access layer. https://nextjs.org/docs/app/guides/authentication
- **Better Auth Expo integration:** Expo app integration and secure session/cookie storage guidance; verify compatibility with pinned SDK and auth versions. https://better-auth.com/docs/integrations/expo
- **Supabase Storage access control:** RLS-based storage access policies and warning that service keys bypass RLS. https://supabase.com/docs/guides/storage/security/access-control
- **Supabase Row Level Security:** Database row-policy setup and testing. https://supabase.com/docs/guides/database/postgres/row-level-security
- **Cloudinary media access control:** Public default delivery distinction, private/authenticated delivery and access-control features. https://cloudinary.com/documentation/control_access_to_media
- **Expo SecureStore:** Local secure storage integration for sensitive session artifacts. https://docs.expo.dev/versions/latest/sdk/securestore/
- **Apple LocalAuthentication:** Platform-mediated biometric/device authentication model. https://developer.apple.com/documentation/localauthentication
- **OWASP MASVS:** Mobile application security verification baseline. https://mas.owasp.org/MASVS/
- **OWASP ASVS:** Web application security verification baseline. https://owasp.org/www-project-application-security-verification-standard/
- **OWASP LLM Top 10:** Common AI application risks such as prompt injection, sensitive information and excessive agency. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/
- **Google Gemini API terms:** Review actual API terms, data handling and differences by service/tier. https://ai.google.dev/gemini-api/terms
- **Anthropic API data retention:** Confirm current API retention terms, exceptions and eligibility for special arrangements. https://privacy.claude.com/en/articles/7996866-how-long-do-you-store-my-organization-s-data
- **Kenya ODPC guidance:** Data-protection guidance; legal applicability needs counsel review. https://www.odpc.go.ke/guidelines-2/
- **ODPC DPIA guidance note:** DPIA preparation context for high-risk processing. https://www.odpc.go.ke/wp-content/uploads/2024/02/ODPC-Guidance-Note-on-Data-Protection-Impact-Assessment-1.pdf
- **Kenya Law - Law of Succession Act, Cap. 160:** Starting point for legal review of will execution; consult the current official consolidated law/counsel. https://kenyalaw.org/kl/fileadmin/pdfdownloads/Acts/LawofSuccessionAct_Cap160.pdf
- **OWASP Logging Cheat Sheet:** Logging security-relevant events while protecting sensitive data. https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

J.3 Review and sign-off checklist

- Product owner approves final MVP scope, screen inventory, role model, supported jurisdictions and inactivity policy.
- Security architect approves threat model, encryption/key recovery, admin separation of duties, object-storage delivery and audit design.
- Qualified legal adviser reviews will storage/disclaimer language, death evidence standards, trustee responsibilities, recipient release, disputed cases, retention and jurisdiction issues.
- Privacy lead completes data inventory, DPIA as required, AI/provider review, retention matrix, data-subject request workflow and processor contracts.

- Engineering lead approves repository structure, Prisma migrations, API contracts, state machine, error model and idempotency strategy.
- QA lead approves requirement traceability, test catalogue, mobile matrix, security negative tests and final regression report.
- Operations lead approves on-call, backup/restore, alerting, incident runbooks, provider failure handling and release rollback.
- Company leadership names admin role holders, independent release approvers, backup approvers and the Super Admin bootstrap/recovery custodian.
- App-store privacy disclosures and public policies match implemented data flows, including optional AI processing.
- No unresolved Critical/High risk is accepted without explicit owner, mitigation, documented rationale and review date.

Implementation sign-off

This document is the proposed technical baseline for implementation planning. It is intentionally explicit about recommended controls while identifying decisions that remain open. It should become the source of truth only after the designated owners approve the architecture decisions, legal/privacy requirements and launch acceptance criteria.

Approval role	Name / title	Decision / comments	Date
Product owner - Pie Corp Limited			
Engineering lead			
Security / cryptography reviewer			
Privacy lead / DPO where applicable			
Qualified legal adviser			
QA / release owner			
Operations owner			

Release boundary No production release of the death-verification or beneficiary-release functionality should occur until the cryptographic recovery model, case policy, two-person authorization, legal evidence requirements and restore procedure have all been implemented and independently tested.